



ARTIFICIAL INTELLIGENCE · HOW COMPUTERS SEARCH

Uninformed Search

How a computer finds an answer without any hint/knowledge.

01



PART ONE

The Basics

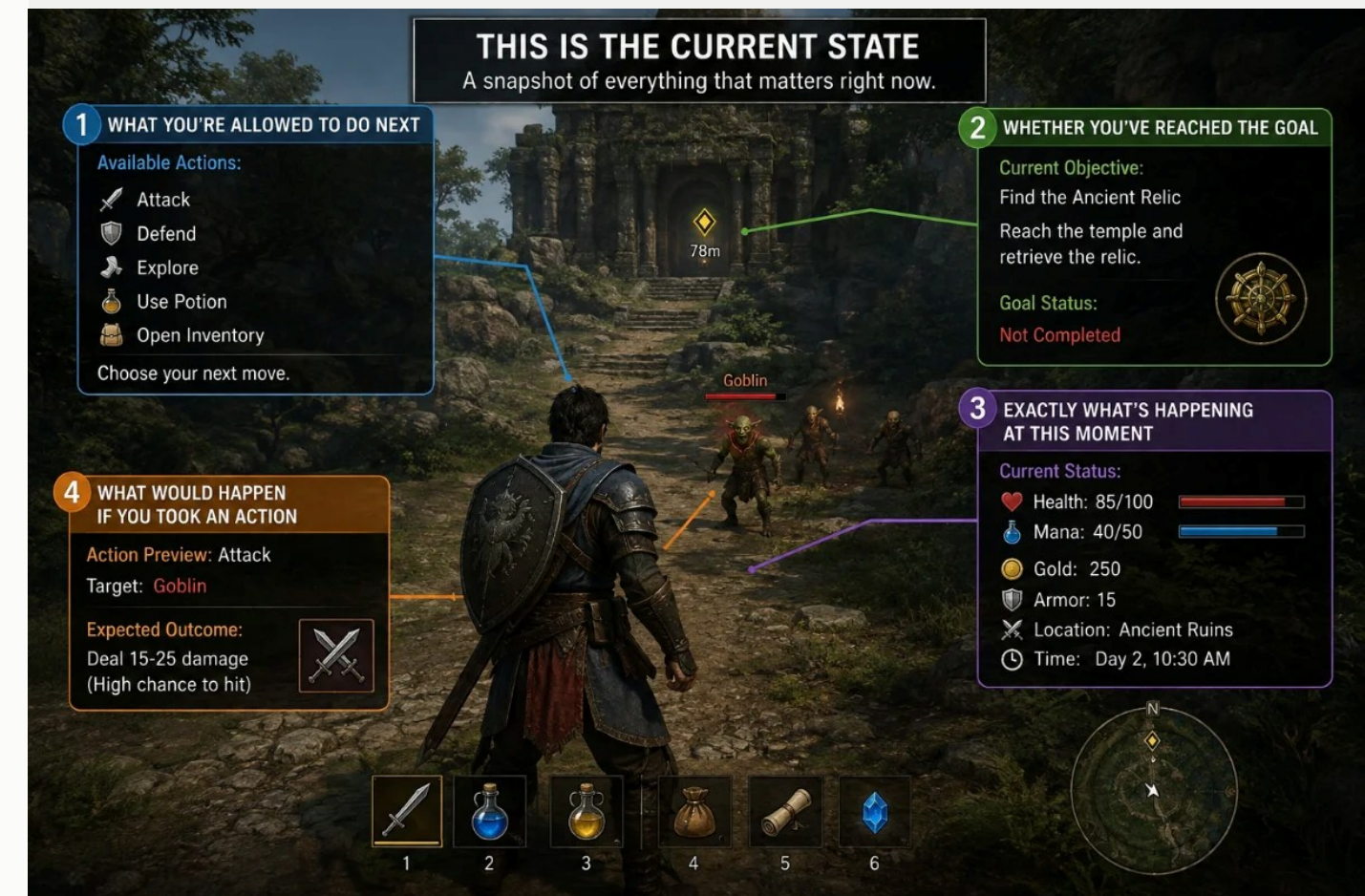
KEY IDEA

What Is a "State"?

Think of a **state** like a screenshot of a video game — it shows everything that matters right now: where you are, what you're holding, what's still left to do.

A good state tells you:

- exactly **what's happening** at this moment
- what you're allowed to do **next**
- what would happen if you took an **action**
- whether you've already reached the **goal**



A state answers one question: "what's the whole picture, right now?"

State Space Search: The Big Picture

A search problem means exploring a space of states to find a path from where you are to where you want to be.

The 4 Parts of Any Search Problem



1. Initial State

where you start



2. Goal State

where you want to reach



3. Actions

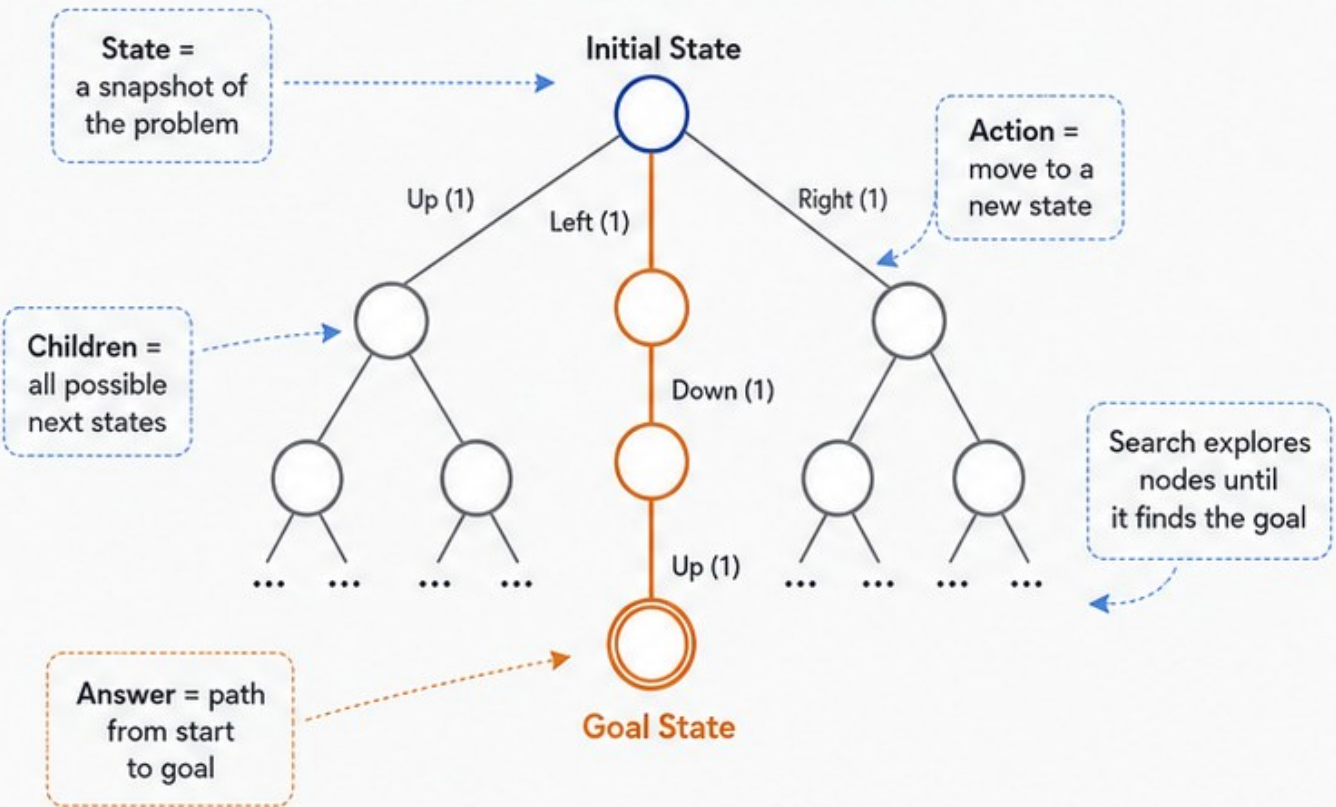
moves you can make from a state



4. Cost

price of each action

How State Space Search Works



Search in 5 Simple Steps

- 1 Start at initial state
- 2 Try possible actions
- 3 Generate new states
- 4 Explore the tree/graph
- 5 Stop when goal is found

Example 1: 8-Puzzle

Initial (start)

1	2	3
4	5	6
7		8

Goal (target)

1	2	3
4	5	6
7	8	



Initial: current tile arrangement



Goal: solved arrangement

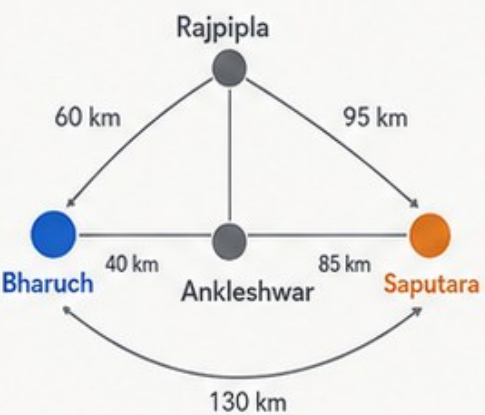


Actions: move blank up / down / left / right



Cost: 1 per move

Example 2: City-to-City



Initial: Bharuch



Goal: Saputara



Actions: go to a connected city



Cost: distance / time



**Different problems,
same search idea.**

Only the definitions of
state, action, and cost change.

A Robot Vacuum's World

Current state: robot in A, A is dirty, B is clean.

It can: suck up the dirt in A, or move to room B.

Counting the States

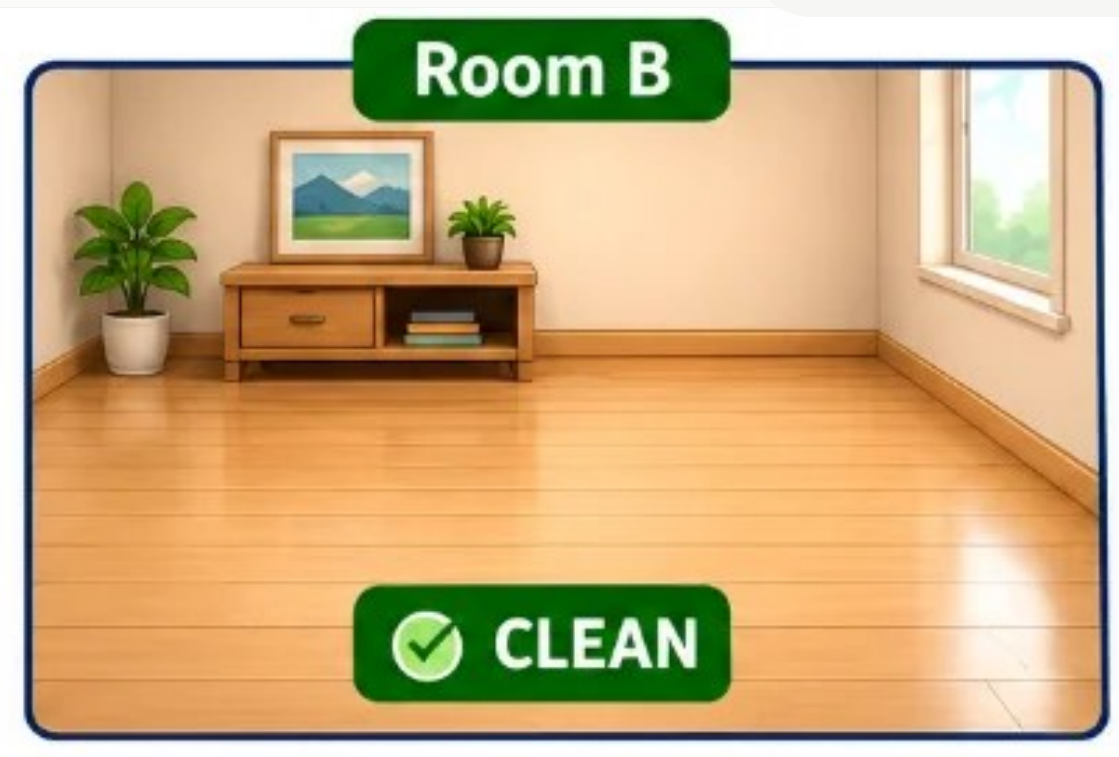
Where's the robot? A or B — 2 choices

Is Room A clean? yes or no — 2 choices

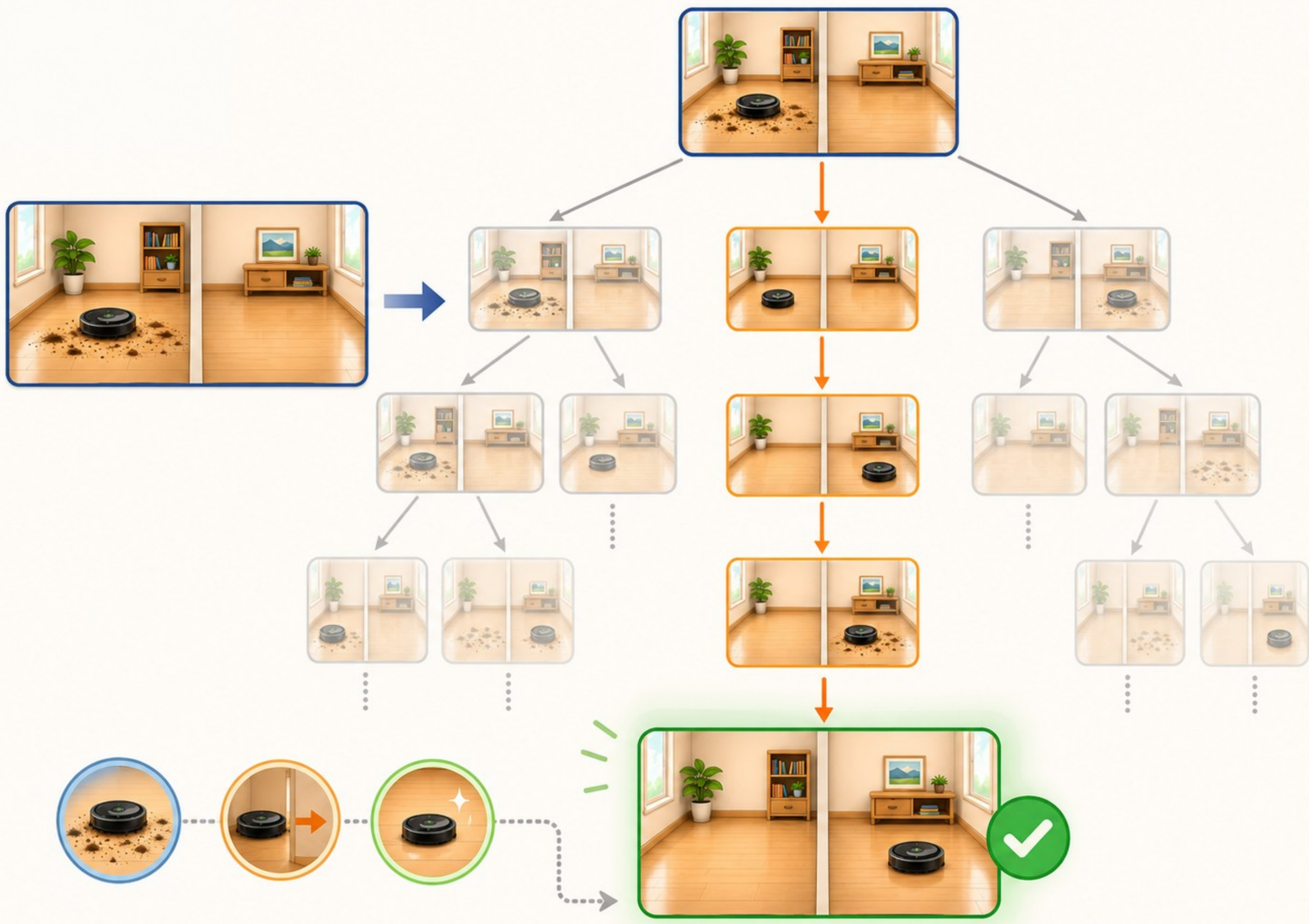
Is Room B clean? yes or no — 2 choices

$2 \times 2 \times 2 = 8$ possible states

The **goal** is any state where both rooms are clean — that's 2 of the 8.



All 8 Possible States			
State	Robot Position	Room A Clean?	Room B Clean?
1	A	Yes	Yes
2	A	Yes	No
3	A	No	Yes
4	A	No	No
5	B	Yes	Yes
6	B	Yes	No
7	B	No	Yes
8	B	No	No



The Sliding Number Puzzle

Start

1		3
4	2	6
7	5	8



Goal

1	2	3
4	5	6
7	8	

You slide tiles into the empty square, trying to line up 1 through 8.

Why It's Hard

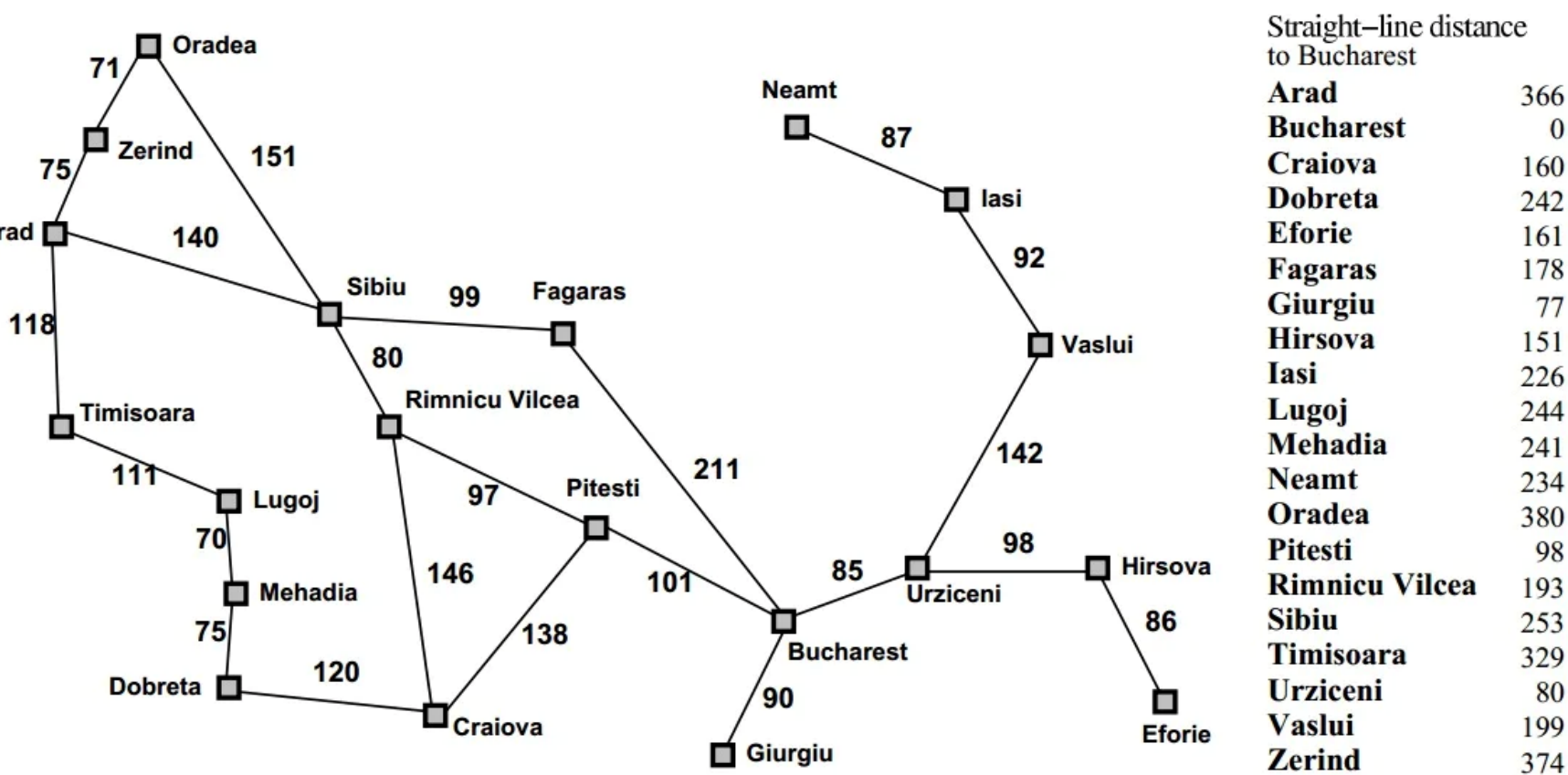
Arrangements: 181,440 different ways to shuffle the tiles

Moves: slide a tile up, down, left, or right

Goal: the fewest slides that solve it

With 181,440 possible arrangements, you can't just try things at random — you need a smart, systematic search.

The Road Trip Problem



Problem: find the shortest road trip from the city of **Arad** to the city of **Bucharest**.

Setting Up the Problem

States: each city (Arad, Sibiu, Bucharest...)

Moves: drive to a city connected by road

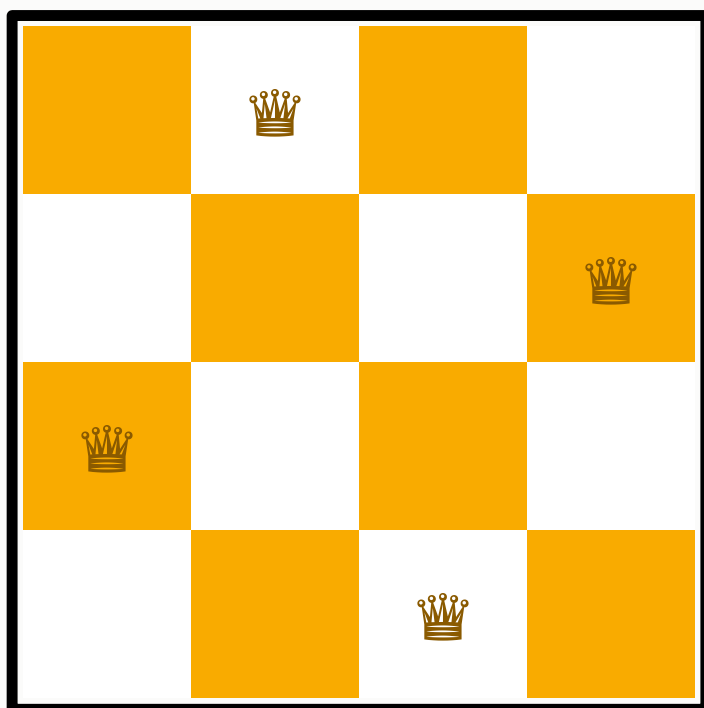
Cost: the distance of that road, in km

Goal: arrive in Bucharest

Several roads lead to Bucharest. A search figures out which combination of roads adds up to the shortest trip.

The Chess Queens Puzzle

Place 4 queens on a 4×4 board so that no queen can capture another.



No queen can capture another ✓

Why It's Tricky

Goal: no two queens share a row, column, or diagonal

Moves: place or shift a queen

4-Queens: only 2 correct arrangements

8-Queens: only 92 correct arrangements

Out of millions of ways to place the queens, only a tiny handful actually work — search finds them without checking every single one.

Five Ways to Evaluate a Search Method

A strategy is really just a rule for which node to explore next. Here's how we grade that rule.

1

Completeness

Will it always find a solution if one exists?

3

Space

How much memory does it need at once?

5

Systematicity

Does it avoid checking the same state twice?

2

Time

How many nodes does it have to check?

4

Optimality

Does it find the best solution, or just any one?

The notation you'll see: **b** = branching factor (children per node) · **d** = depth of the best solution · **m** = maximum possible depth (sometimes infinite)

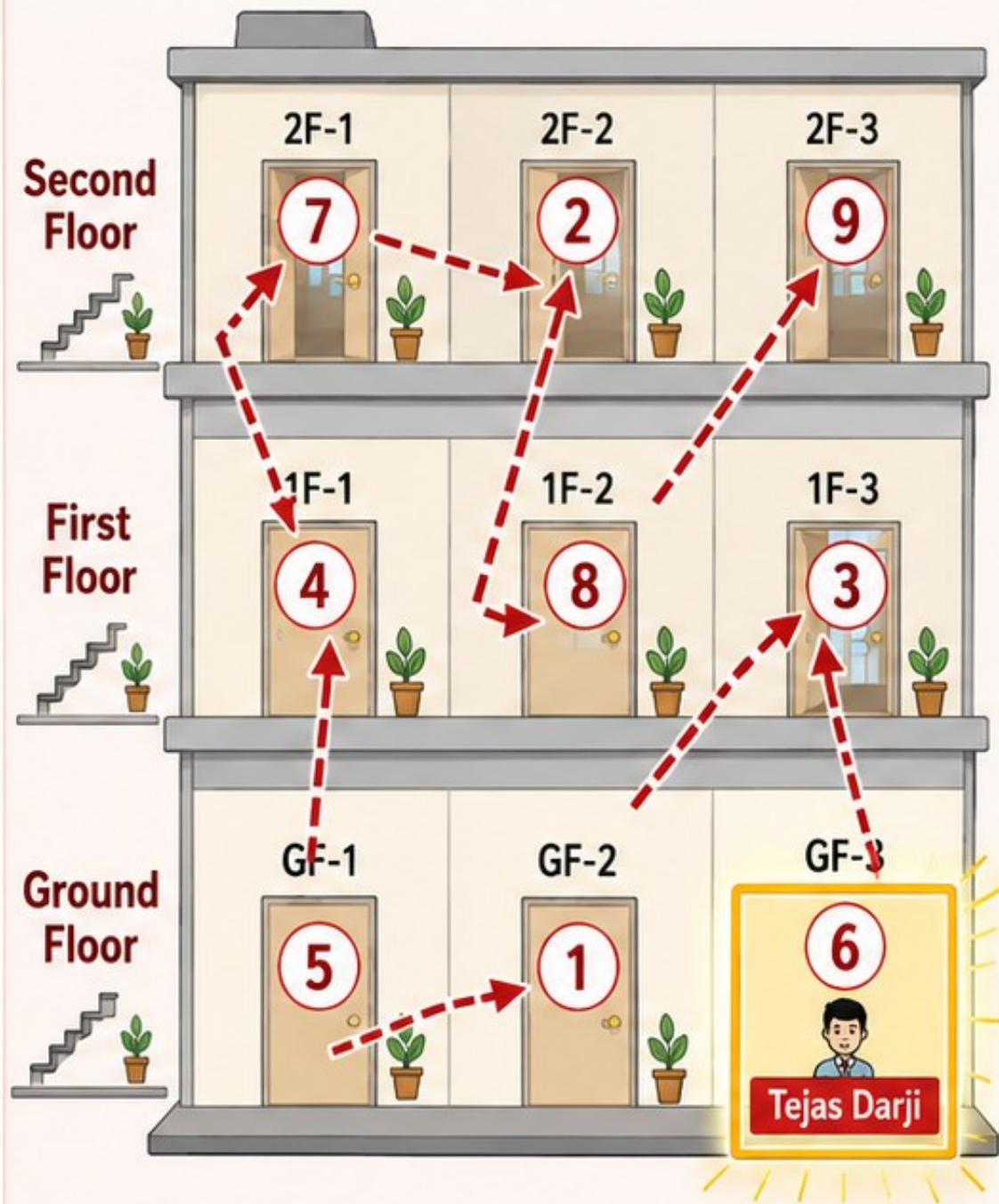
Five Classic Strategies

These are called **uninformed** , or "blind," searches — the computer can't tell if one path looks more promising than another. It just follows a rule.

- A** **Breadth-First Search** — check everything close by, one ring out at a time
- B** **Depth-First Search** — pick one path and follow it as far as it goes
- C** **Depth-Limited Search** — like DFS, but with a maximum depth so it doesn't wander forever
- D** **Iterative Deepening** — try a depth limit, then slowly raise it until it works
- E** **Bidirectional Search** — search from the start and the goal at the same time

Brute Force

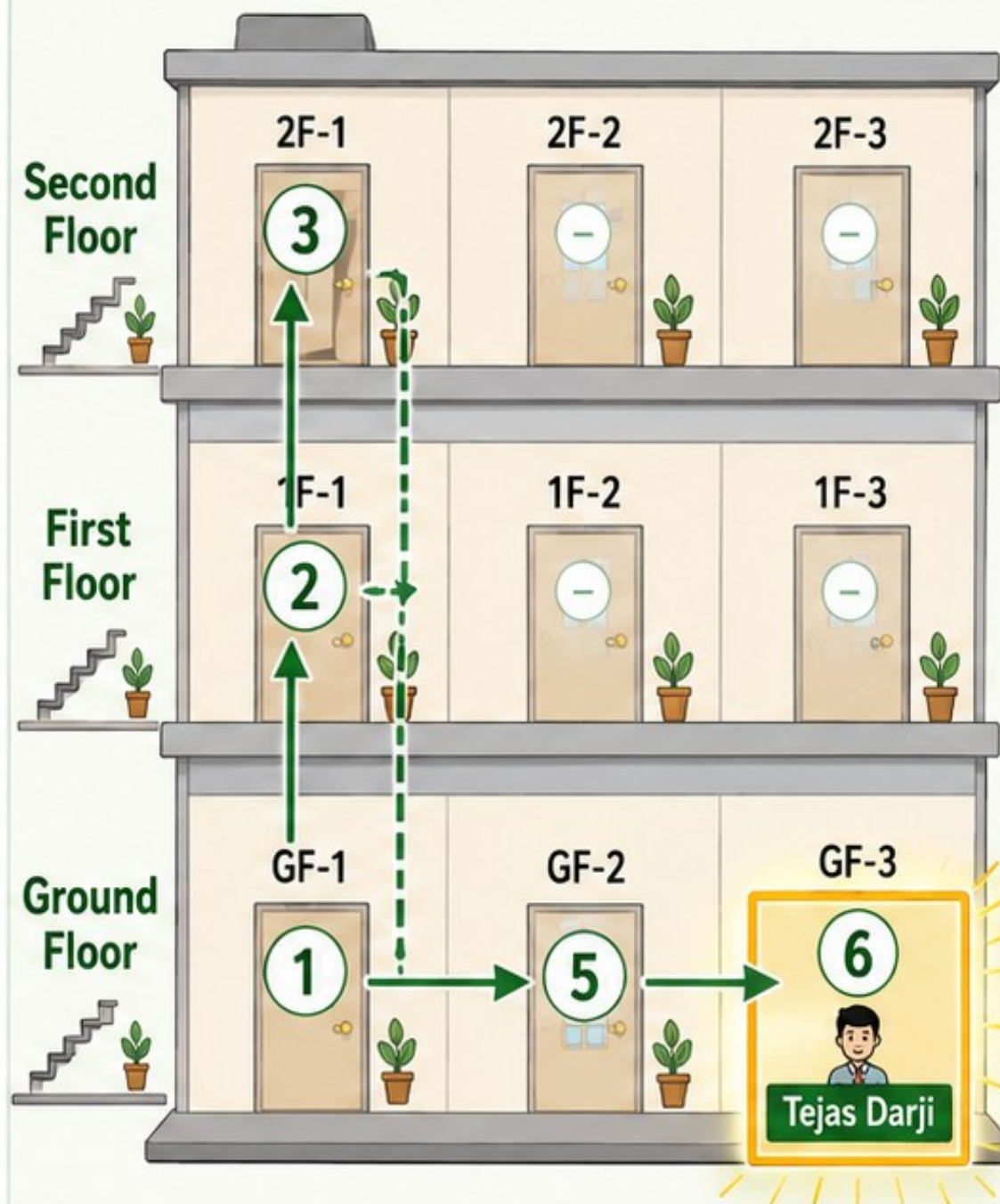
random checking



Unorganized search. Jumps randomly between rooms and floors. Inefficient and slow.

DFS

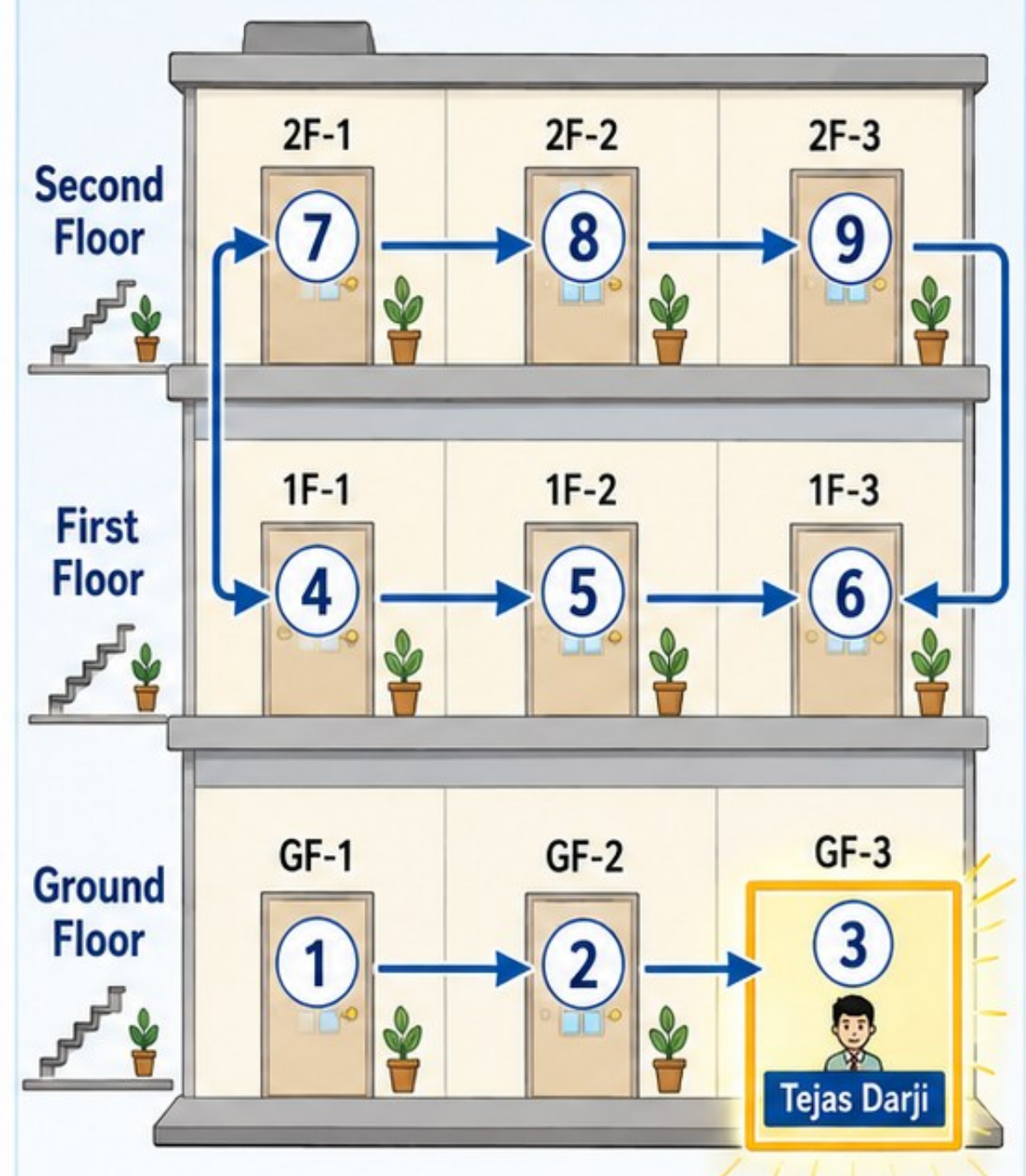
go deep, then backtrack



Go as deep as possible along one path. Backtrack when a dead end is reached, then try the next path.

BFS

level by level



Explore all nodes at the current level first, then move to the next level. Finds the target with the fewest steps in unweighted graphs.

Breadth-First Search: Level by Level



Order: $S \rightarrow A, B, C \rightarrow D, E, F, G, H \rightarrow \dots$

What BFS Is Like

BFS explores **all nearby places first**, then moves one step farther.

Think of it like **ripples spreading in water** after you throw a stone.

Complete: Yes. If a path exists, BFS will eventually find it.

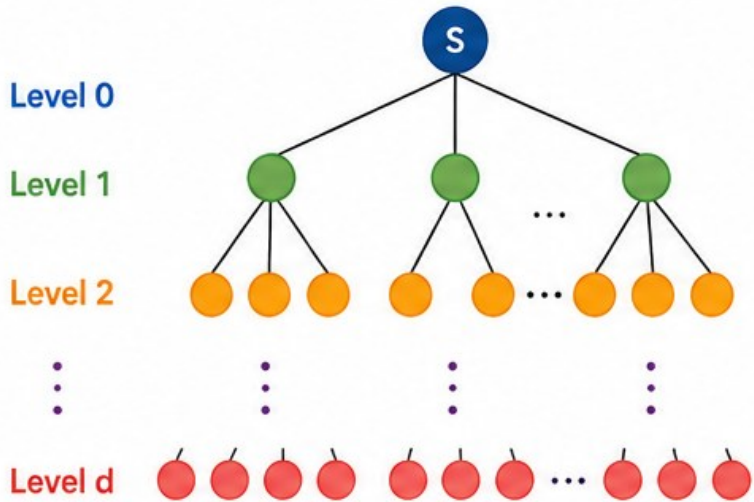
Optimal: Yes, when every move has the same cost (each step counts equally).

Time / Space: $O(b^d)$ both — it has to remember an awful lot

BFS always finds the shortest path — but it can eat up a huge amount of memory doing it.

Breadth-First Search: Level by Level

TIME COMPLEXITY OF BFS: $O(b^d)$



Assume: branching factor = b (each node has b children)
Goal is at depth = d

Level	Nodes at that level	Total nodes up to that level
0	1	1
1	b	$1 + b$
2	b^2	$1 + b + b^2$
3	b^3	$1 + b + b^2 + b^3$
...	...	...
d	b^d	$1 + b + b^2 + \dots + b^d$

Total nodes visited $\approx 1 + b + b^2 + \dots + b^d = O(b^d)$

TIME COMPLEXITY: $O(b^d)$



BFS visits every node level by level.



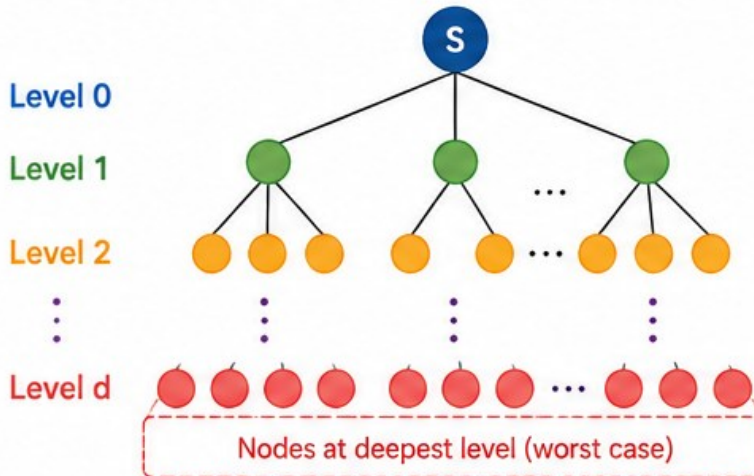
Number of nodes grows exponentially with depth.



More branching (b) and greater depth (d) $\Rightarrow$ much more time.

Time grows exponentially with depth and branching factor.

SPACE COMPLEXITY OF BFS: $O(b^d)$



 BFS stores all nodes of the current level in the queue. Worst case: it stores all nodes at the deepest level.

Level	Nodes stored in queue (max)	Why?
0	1	Only root
1	b	All nodes at level 1
2	b^2	All nodes at level 2
3	b^3	All nodes at level 3
...	...	...
d	b^d	All nodes at deepest level

Maximum nodes in memory $\approx b^d = O(b^d)$

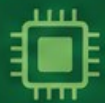
SPACE COMPLEXITY: $O(b^d)$



BFS keeps all nodes of the current level in the queue.

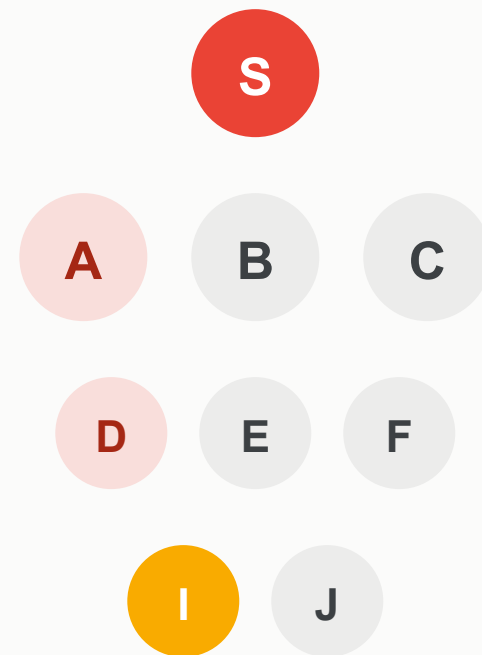


At the deepest level, up to b^d nodes may be stored at once.



Space required grows exponentially with depth and branching factor.

Depth-First Search: All the Way Down



Order: $S \rightarrow A \rightarrow D \rightarrow I \rightarrow J \rightarrow \dots$ (backtrack) $\rightarrow E \rightarrow \dots$

What DFS Is Like

Commits to one path and follows it until it's a dead end, then backs up

Complete: no — could get stuck down an endless path

Optimal: no — finds a solution, not necessarily the best one

Time: $O(b^m)$ · **Space:** $O(bm)$ — only remembers the current path

DFS uses way less memory than BFS — but it might never find the best answer.

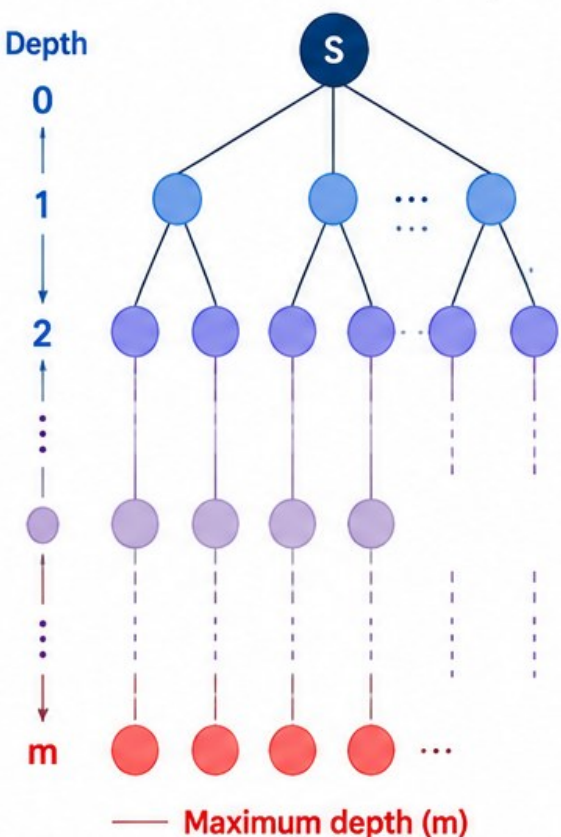
Depth-First Search: All the Way Down

TIME COMPLEXITY OF DFS: $O(b^m)$

DFS may explore many paths.

b = branching factor (avg. number of children)

m = maximum depth of the search tree



Depth Level	Nodes at that level	Possible nodes up to that level
0	1	1
1	b	$1 + b$
2	b^2	$1 + b + b^2$
3	b^3	$1 + b + b^2 + b^3$
...	...	...
m	b^m	$1 + b + b^2 + \dots + b^m$



Total nodes DFS may visit

$$= 1 + b + b^2 + \dots + b^m$$

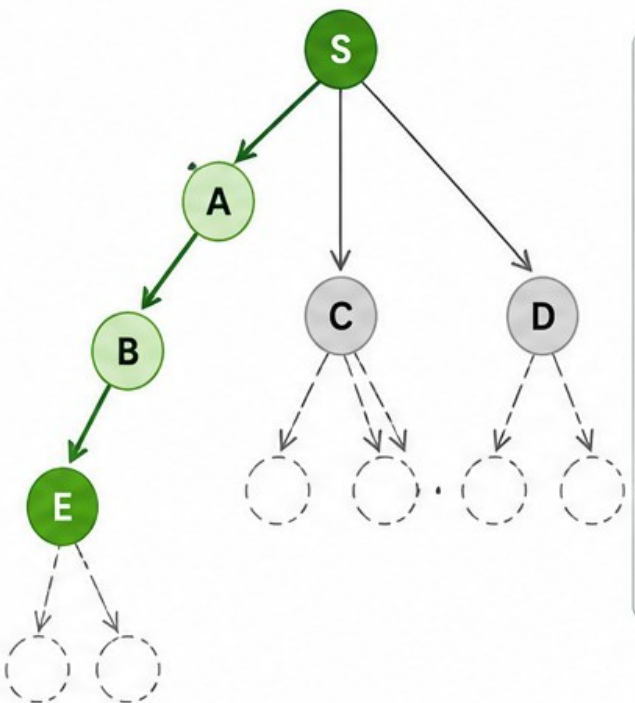
$$\approx O(b^m)$$



More branching (b) and greater depth (m)
→ more time (exponential growth)

SPACE COMPLEXITY OF DFS: $O(bm)$

DFS stores only the current path from root to the deepest node it is exploring.



MEMORY (RECURSION STACK)

S (depth 0)
A (depth 1)
B (depth 2)
E (depth 3)
...

At most $m + 1$ nodes stored



Maximum nodes stored at any time = $m + 1$
Each level needs space for up to b children.

$$\text{SPACE COMPLEXITY} = O(bm)$$



Stores only one path



Uses much less memory than BFS



Good for deep searches

When BFS Gets Too Big to Run

Assuming 10 choices at each step, 1 million nodes checked per second, 1000 bytes per node.

Depth	Nodes checked	Time	Memory
2	110	milliseconds	11 KB
4	11,110	milliseconds	11 MB
6	10^6	seconds	1.1 GB
8	10^8	minutes	100 GB
10	10^{10}	hours	10 TB
12	10^{12}	days	1 PB
14	10^{14}	years	100 PB
16	10^{16}	years	10 EB

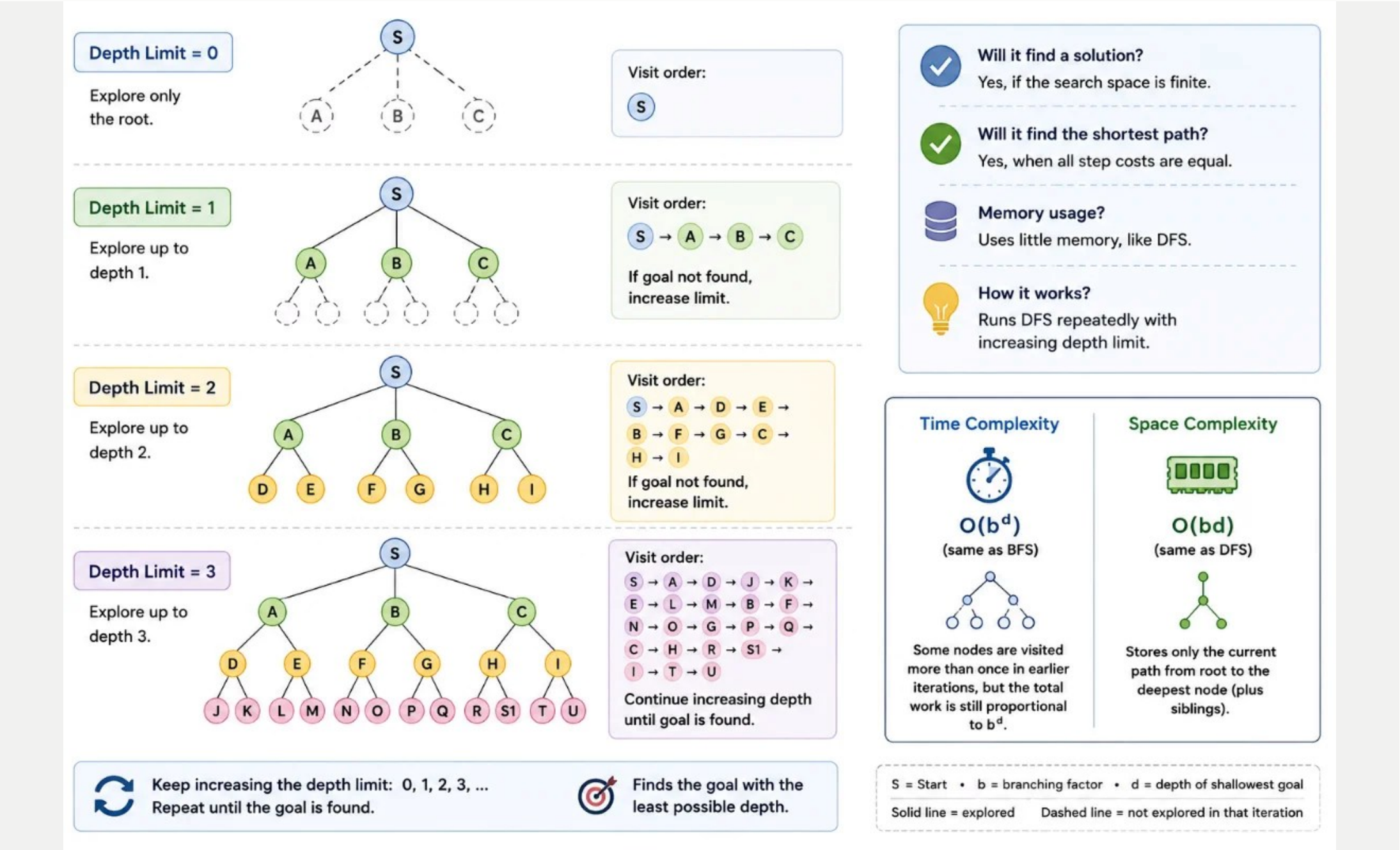
Notice memory runs out long before time does — that's why we need smarter, more memory-friendly strategies for deep problems.

Iterative Deepening: Try a Little Deeper Each Time

How It Works

1. Try DFS, but stop at depth 0
2. Didn't find it? Try again, stop at depth 1
3. Keep raising the limit until the goal turns up

It repeats some work, but most of the effort happens at the deepest level anyway — so the repeats barely cost anything. Complete, optimal, and only needs $O(bd)$ memory.



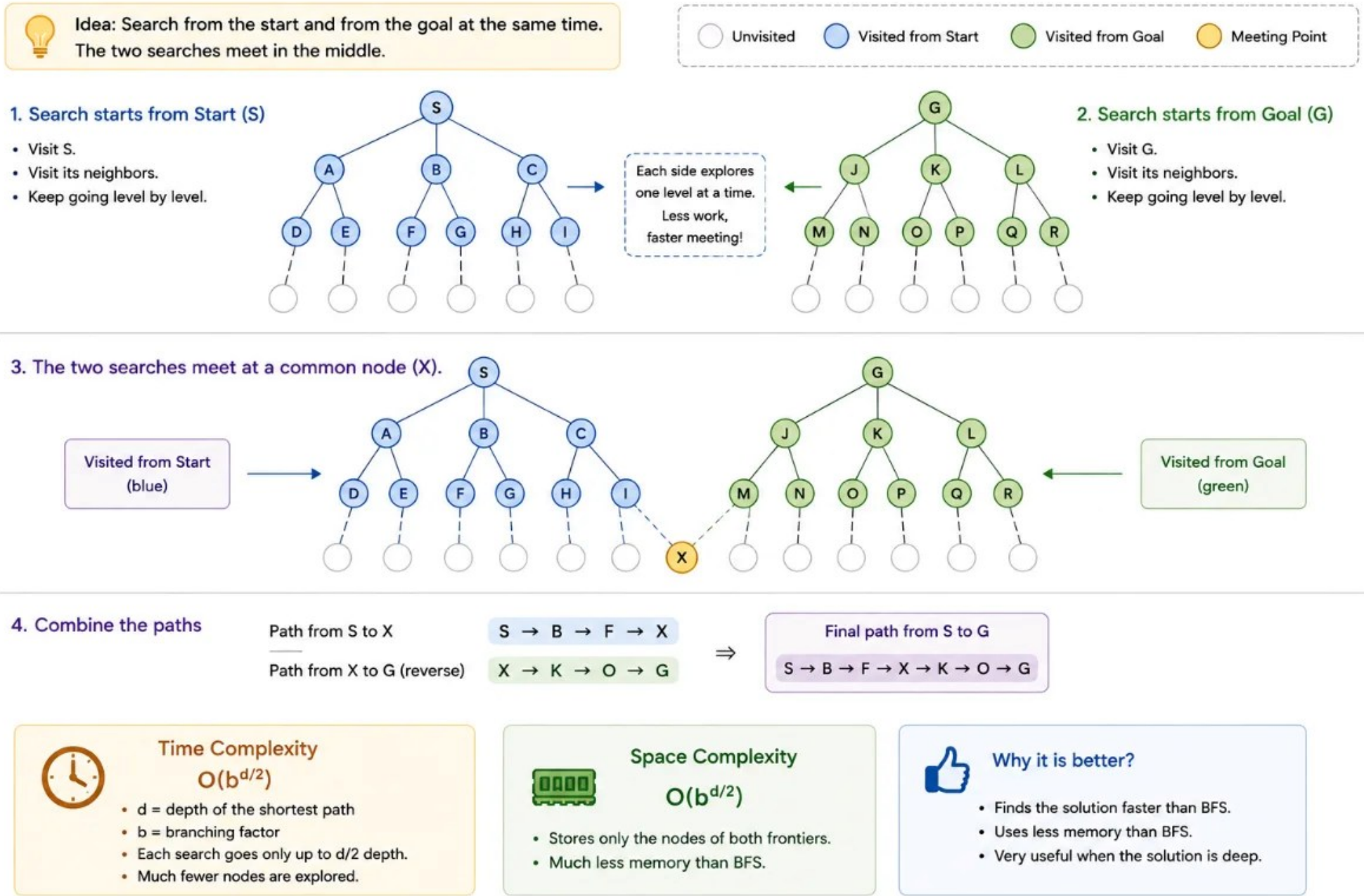
Goal found at depth 2 ✓

Bidirectional: Meet in the Middle

Why It's So Fast

Two shorter searches instead of one long one
Stops the instant the two searches meet
Needs moves that work in reverse, and a known goal

$O(b^d)$ becomes roughly $O(b^{(d/2)})$ — for $d = 10$ that's the difference between 10 billion checks and about 200,000.



The Two Best All-Rounders

Iterative Deepening

The best general-purpose choice

Combines BFS's completeness with DFS's low memory use

Complete and optimal — space stays at just $O(bd)$

Reach for it when you don't know how deep the answer is and want the best one, on limited memory.

Bidirectional Search

The fastest choice, when it applies

$O(b^d)$ shrinks to roughly $O(b^{d/2})$

Needs reversible moves and a known goal state

Reach for it when you can work backward from the goal and the search space is huge.

Choosing the Right Strategy

Pick BFS when...

the answer is shallow, you want the best one, and memory isn't tight

Pick DFS when...

memory is very limited and any solution will do

Pick Iterative Deepening when...

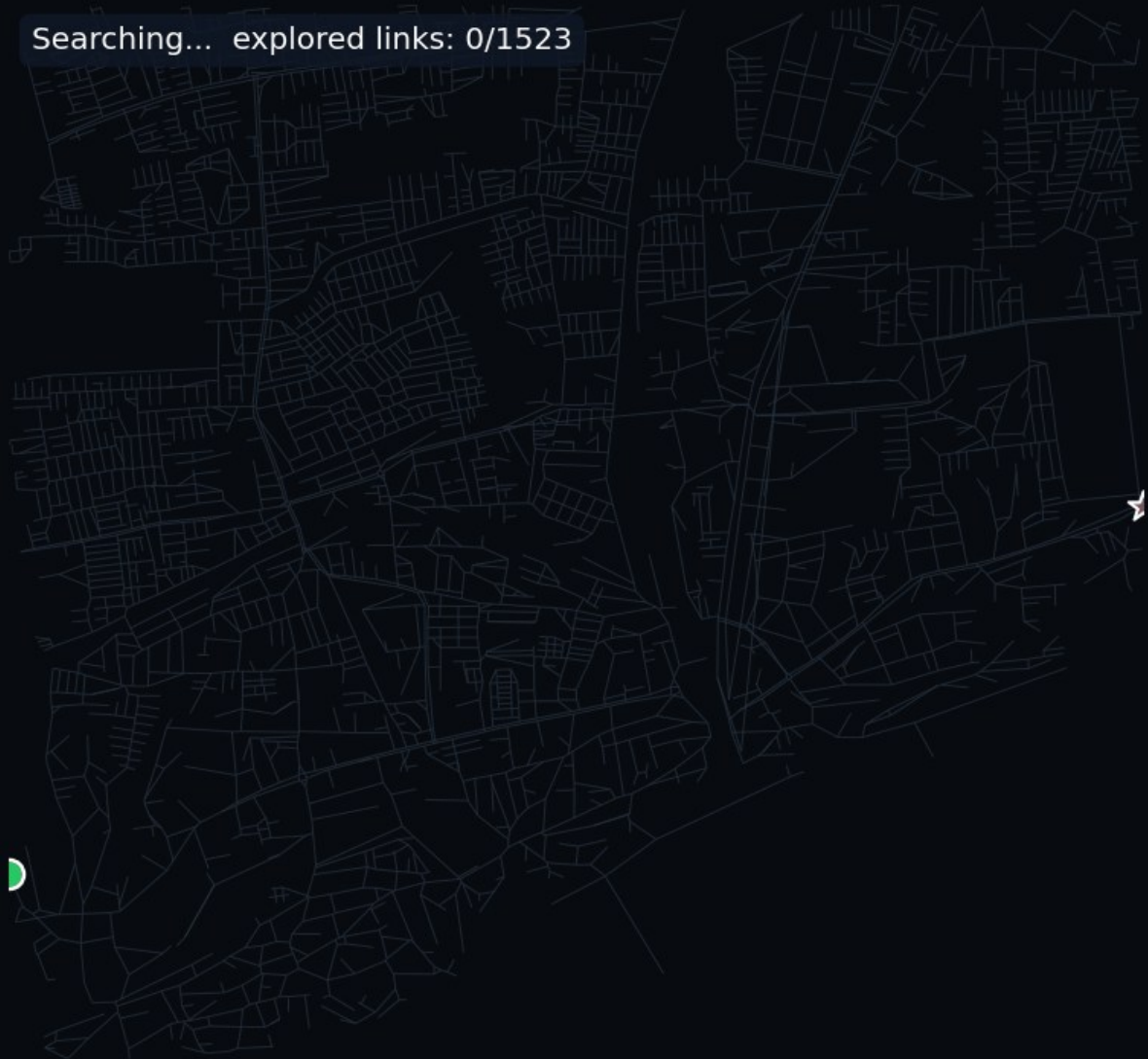
you don't know the depth and want BFS quality on DFS memory

Big idea: uninformed search is all about trading off time, memory, and finding the best answer. Next, we'll see how giving the computer a hint — domain knowledge — lets it do much better.

DFS

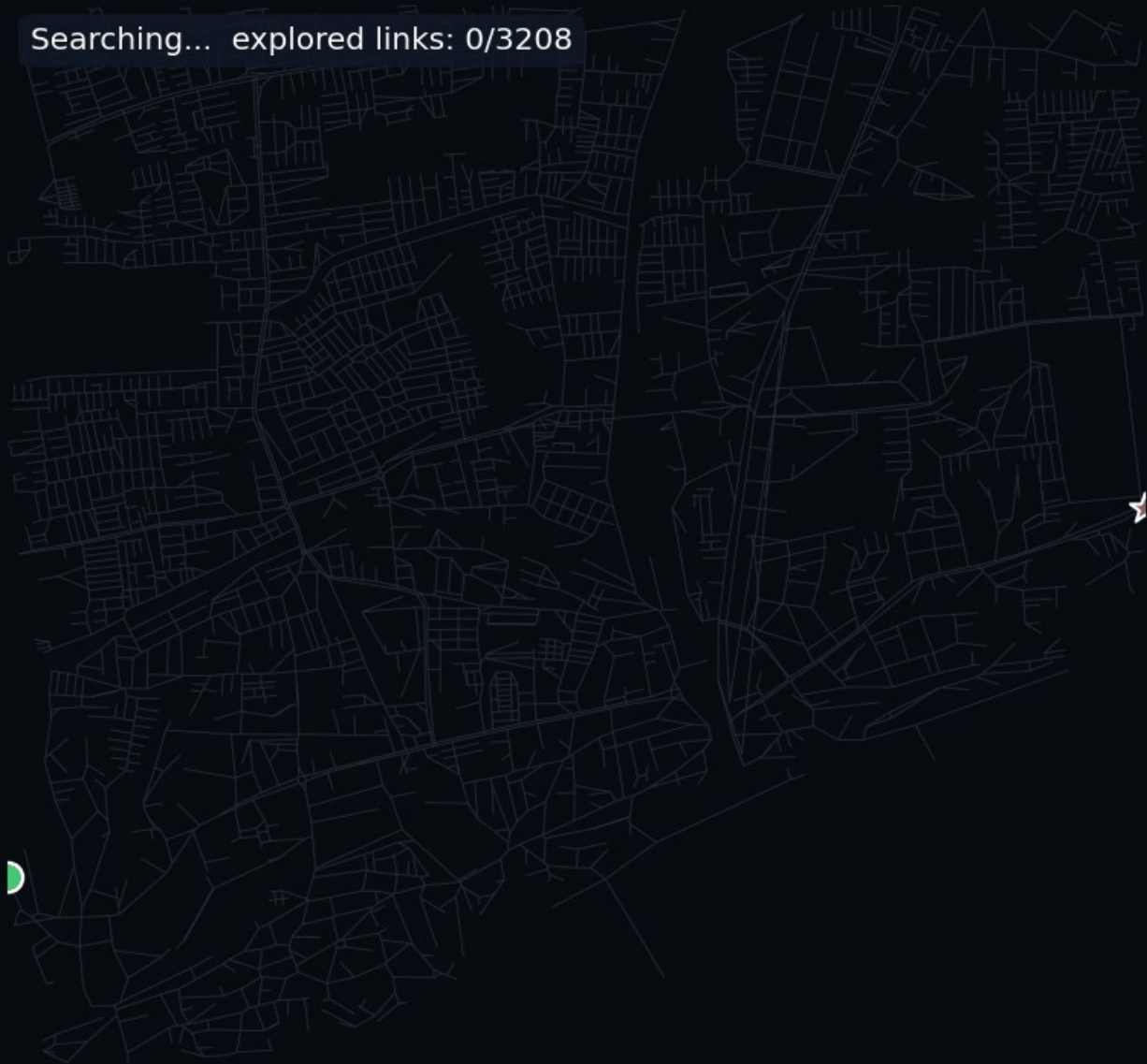
DFS SEARCH ON REAL STREETS

Searching... explored links: 0/1523



BFS SEARCH ON REAL STREETS

Searching... explored links: 0/3208





ARTIFICIAL INTELLIGENCE · MAKING SEARCH SMARTER

Informed Search

Giving the computer a hint so it can search smarter, not just harder.

01



PART ONE

The Search Problem

What Is a Search Problem?

- Imagine planning a road trip to another city.
- You start somewhere,
- You want to end up somewhere else, and
- Every step along the way costs time or fuel.



Search problems are everywhere — GPS directions, puzzle games, robot planning, even your own daily schedule.

Blind vs. Informed Search

BLIND SEARCH

Checks everything, in order

It explores systematically but has no idea which direction is more promising — like checking every drawer in the house one by one.



Examples: breadth-first, depth-first, uniform-cost search. Can be very slow.

INFORMED SEARCH

Uses a hint to guess where to look

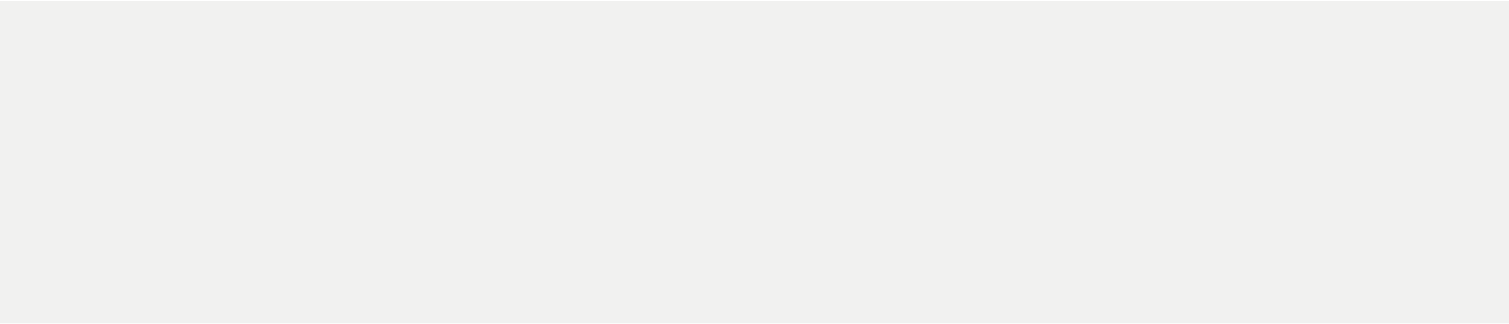
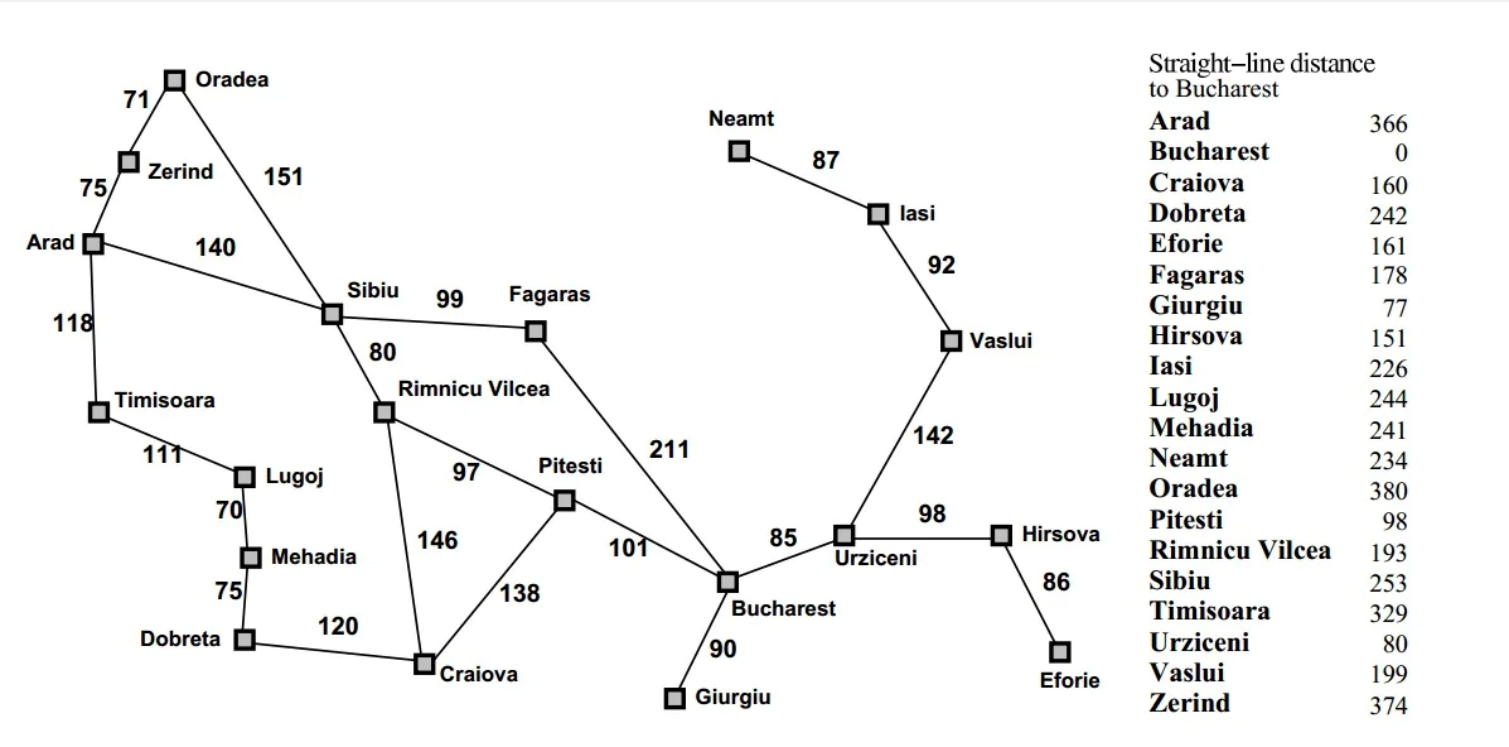
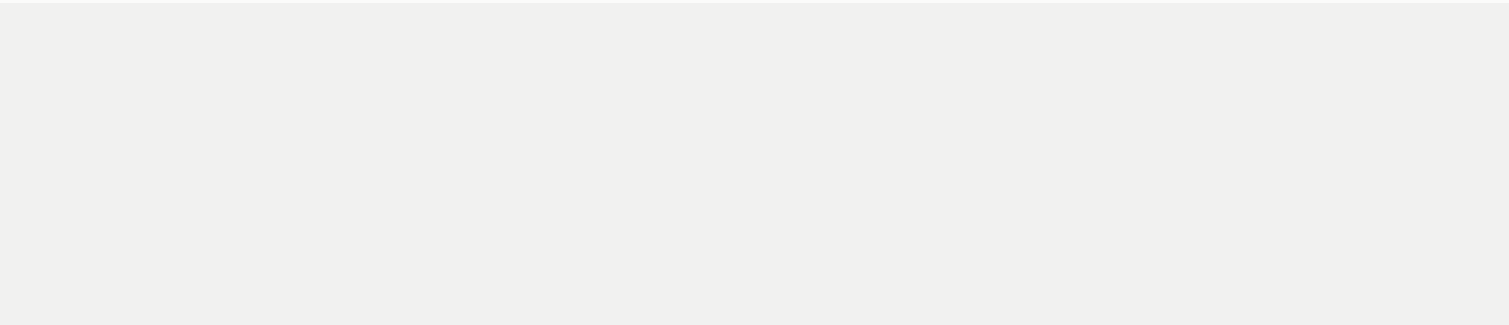
It uses extra knowledge about the problem to guide the search toward the goal — like remembering you usually leave your keys by the door.



Examples: greedy best-first search, A* search. Much faster and more efficient.

What's a Heuristic?

A **heuristic** gives a computer a bit of "intuition" about how close it is to the goal — a smart guess, not a certainty.



Without a Hint, You Can Get Stuck

Standing in Iasi, wanting to reach Oradea: the road to Neamt (87 km) looks shorter than the road to Vaslui (92 km) — so a blind search picks Neamt. But Neamt only leads back to Iasi!

Iasi → Neamt → Iasi → Neamt → ... forever, going nowhere.

A Heuristic Breaks the Loop

A* adds up the distance already traveled *plus* a straight-line guess to the goal. After bouncing back from Neamt, that total climbs to 174 — so Vaslui (92) suddenly looks better, and the search moves on.

$f(n) = g(n) + h(n)$ — cost so far, plus a guess of what's left. It keeps the search moving forward instead of looping.

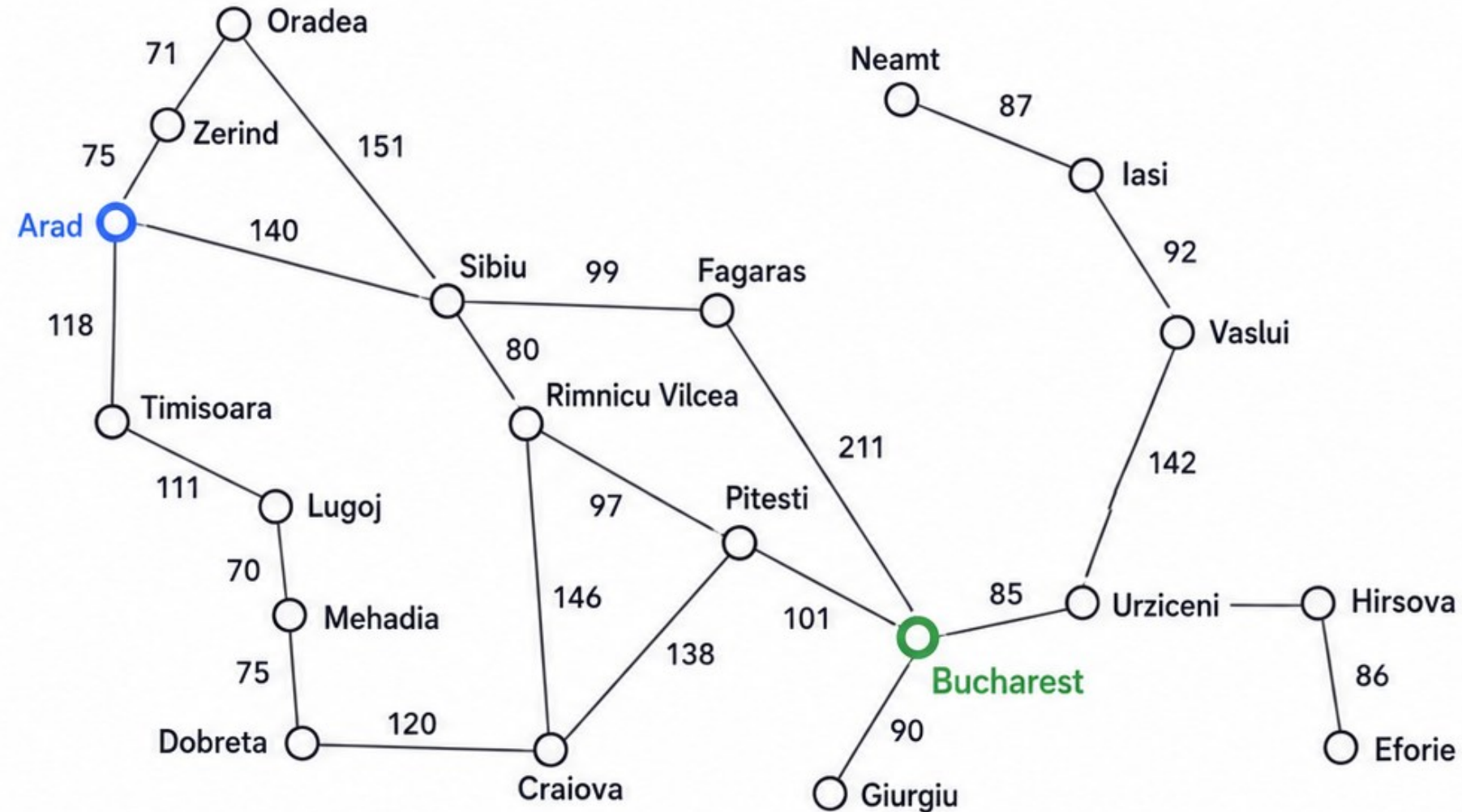
02



PART TWO

The A* Algorithm

The Road Trip Problem



Setting Up the Problem

States: each city (Arad, Sibiu, Bucharest...)

Moves: drive to a city connected by road

Cost: the distance of that road, in km

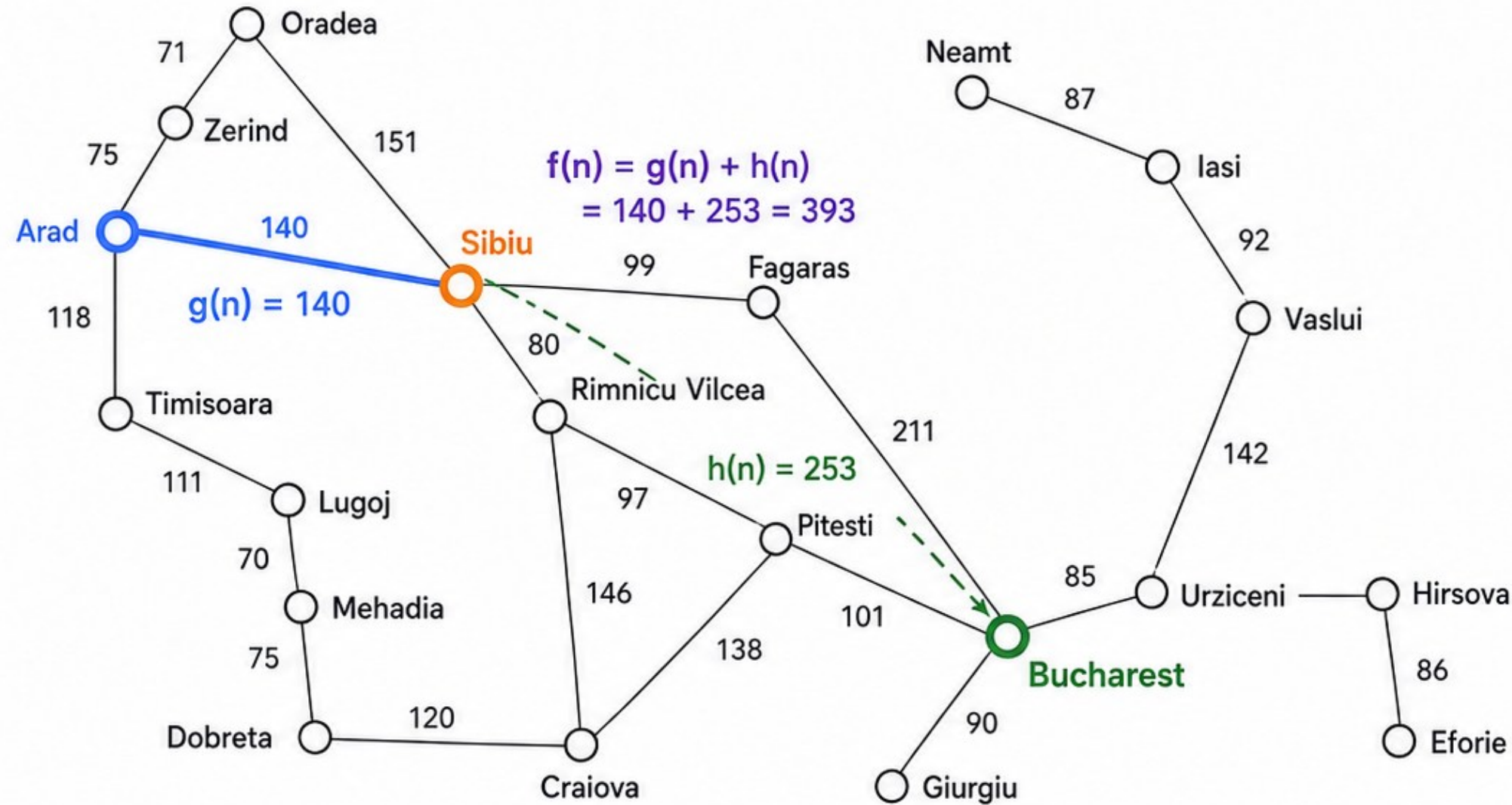
Goal: arrive in Bucharest

Several roads lead to Bucharest. A search figures out which combination of roads adds up to the shortest trip.

Problem: find the shortest road trip from the city of **Arad** to the city of **Bucharest**.

Understanding $g(n)$, $h(n)$ and $f(n)$

At any city, A* looks at three values.



$g(n)$

Distance already traveled to reach this city

Here: 140 km

$h(n)$

Estimated distance left to Bucharest

Here: 253 km

$f(n)$

Total estimated trip cost

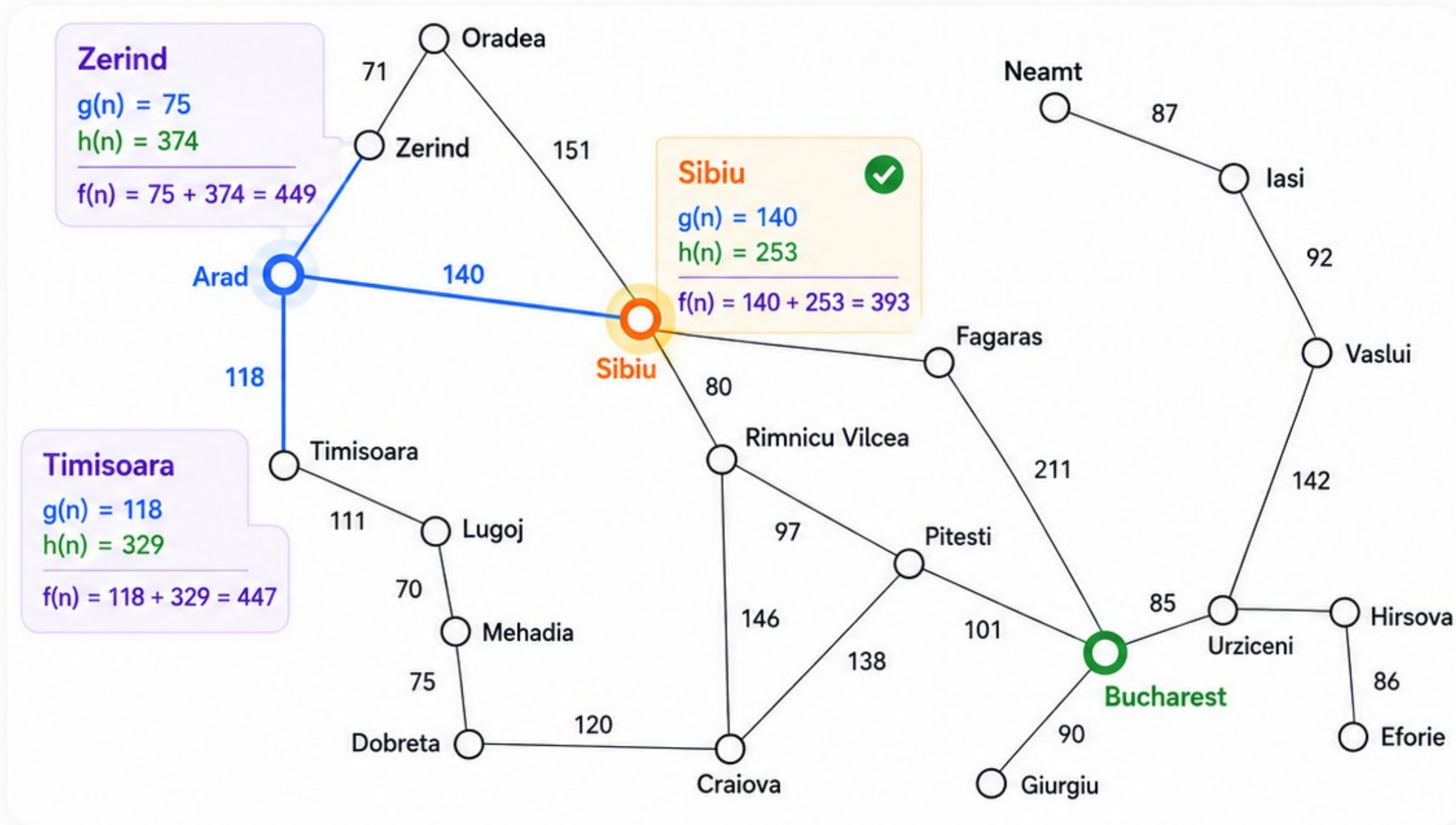
Here: 393 km

A* combines what you have spent so far and what you estimate is left:

$$f(n) = g(n) + h(n).$$

A*'s First Decision from Arad

From Arad, A* checks all neighboring cities and chooses the one with the lowest total estimated cost $f(n)$.



Compare $f(n)$ values

- 1 Sibiu 393
- 2 Timisoara 447
- 3 Zerind 449

★ A* picks **Sibiu** because **393** is the smallest $f(n)$.

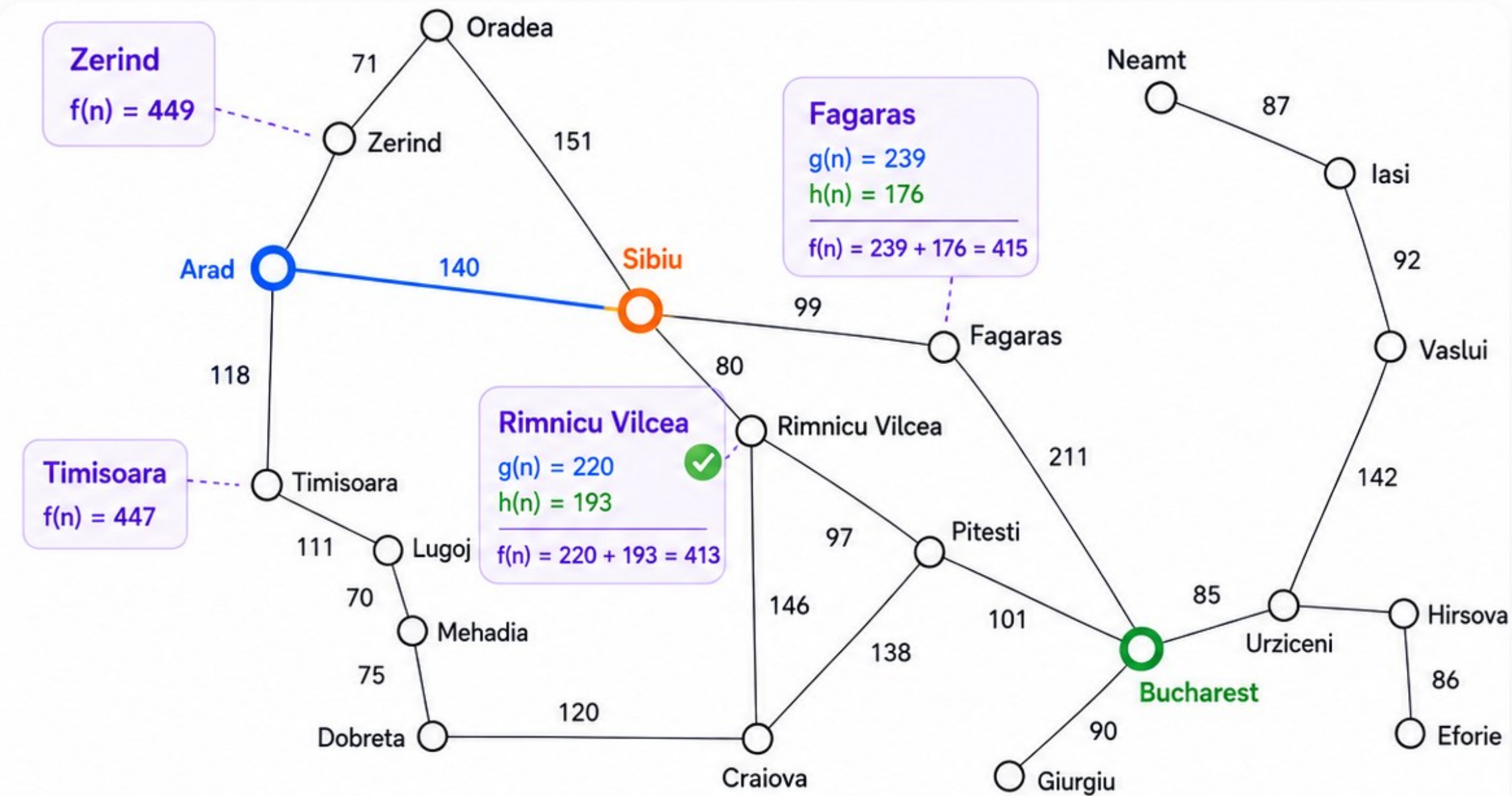
It does not choose the shortest immediate road; it chooses the lowest estimated total trip cost.



Decision: From Arad, A* evaluates its three neighbors and selects **Sibiu** because it has the lowest total estimated cost $f(n) = 393$.

After Sibiu: What does A* consider next?

A* keeps all discovered choices and expands the one with the lowest total estimated cost $f(n)$.



Open List / Candidates (by $f(n)$)

- 1 **Rimnicu Vilcea** 413 ✓
- 2 Fagaras 415
- 3 Timisoara 447
- 4 Zerind 449



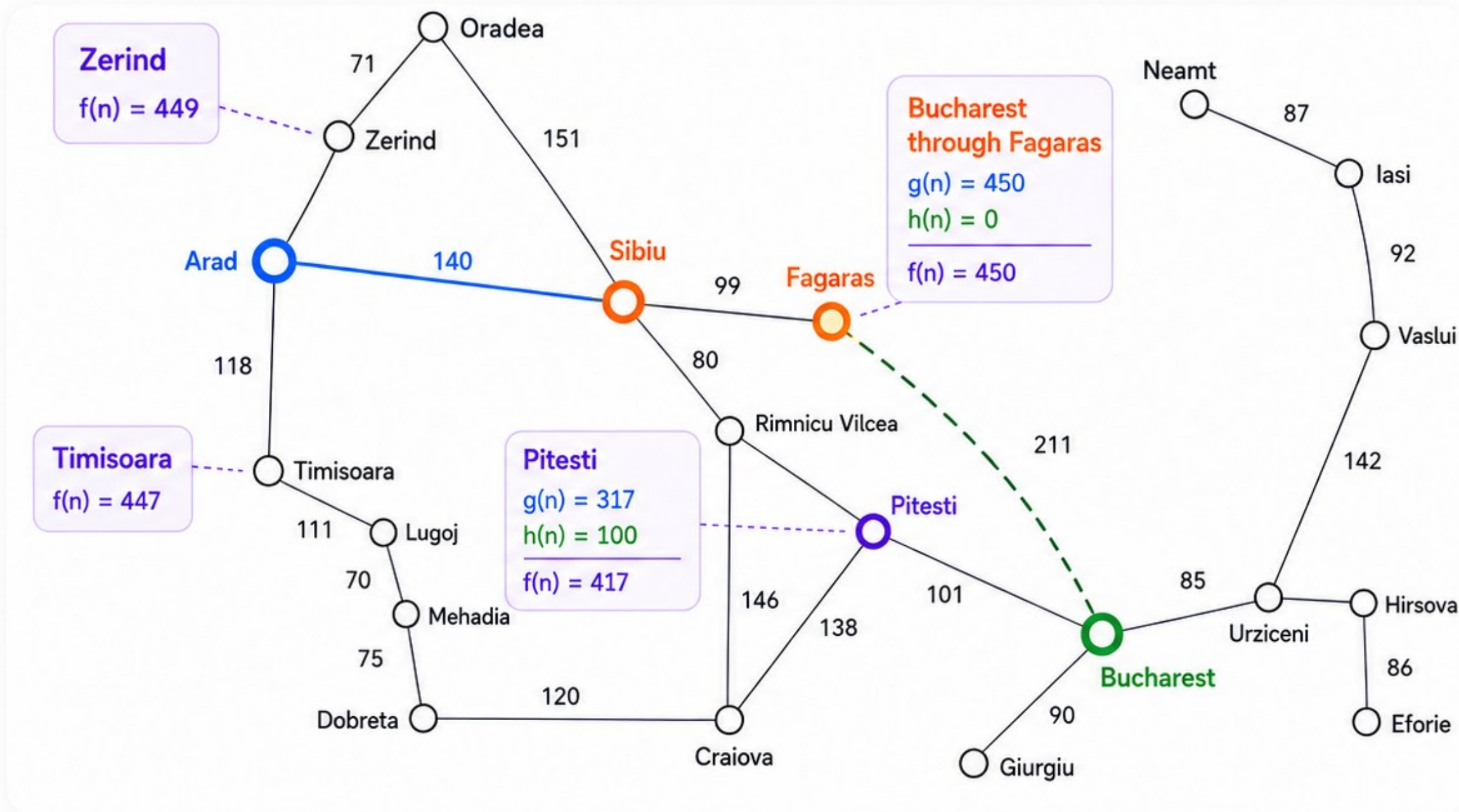
A* does not forget older options. It compares every discovered city in the open list and chooses the one with the smallest $f(n)$.



Decision: After expanding **Sibiu**, A* chooses **Rimnicu Vilcea** next because **413** is the lowest $f(n)$.

Why A* does not stop at Bucharest yet

After expanding Fagaras, A* sees a route to Bucharest — but it still compares all open options and chooses the one with the lowest $f(n)$.



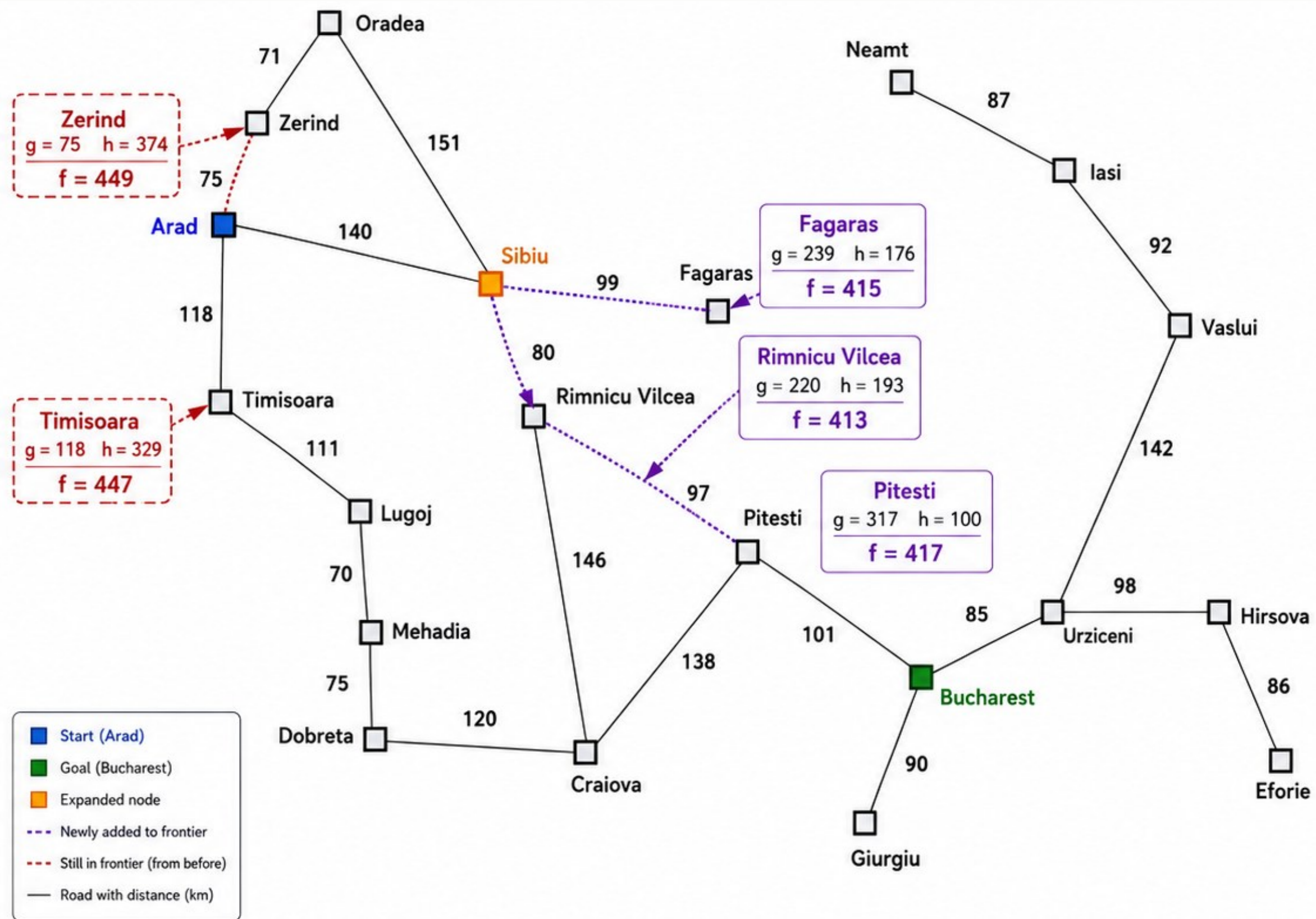
Open List / Candidates

- | | | | |
|---|-----------------------|-----|---|
| 1 | Pitesti | 417 | ✓ |
| 2 | Timisoara | 447 | |
| 3 | Zerind | 449 | |
| 4 | Bucharest via Fagaras | 450 | |

★ A* does not stop the first time it reaches the goal. It stops only when the goal has the lowest $f(n)$ among all open options. Since **417** is less than **450**, **Pitesti** must be explored first.



Decision: A* chooses **Pitesti** next because $f(\text{Pitesti}) = 417$ is smaller than the first goal route $f(\text{Bucharest via Fagaras}) = 450$.



OPEN LIST (Frontier) after expanding Sibiu

Rank	City	$g(n)$	$h(n)$	$f(n) = g + h$
1	Rimnicu Vilcea	220	193	413
2	Fagaras	239	176	415
3	Pitesti	317	100	417
4	Timisoara	118	329	447
5	Zerind	75	374	449

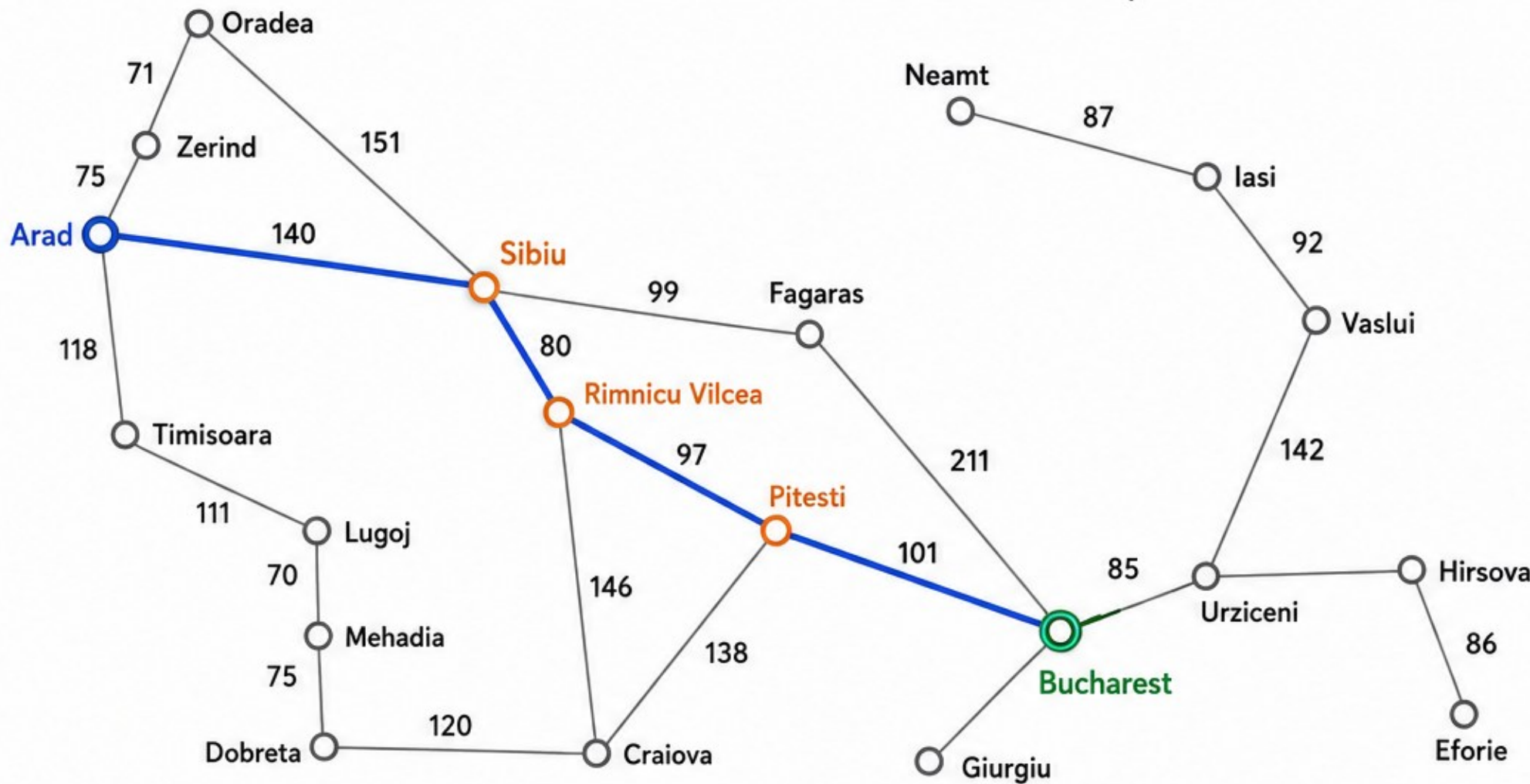


A* always selects the city with the **lowest $f(n)$** . Therefore, the next city to expand is **Rimnicu Vilcea ($f = 413$)**.

Key Idea

A* does not go straight to the goal. It **keeps all possible good options** in the frontier and always picks the most promising one (lowest f).

A* finds the lowest total cost path from Arad to Bucharest.



Best Path (Found by A*)

Arad

↓ 140

Sibiu

↓ 80

Rimnicu Vilcea

↓ 97

Pitesti

↓ 101

Bucharest (Goal)

Total Cost = 418 km

Why not the other path?

A* does not stop when it first reaches the goal. It always expands the node with the lowest $f(n)$. Although Fagaras reaches Bucharest, its total cost (f) is higher than the path through Pitesti.

Alternative (Not chosen)

Arad → Sibiu → Făgaras → Bucharest

Arad

140

Sibiu

99

Fagaras

211

Bucharest

Total Cost = 450 km

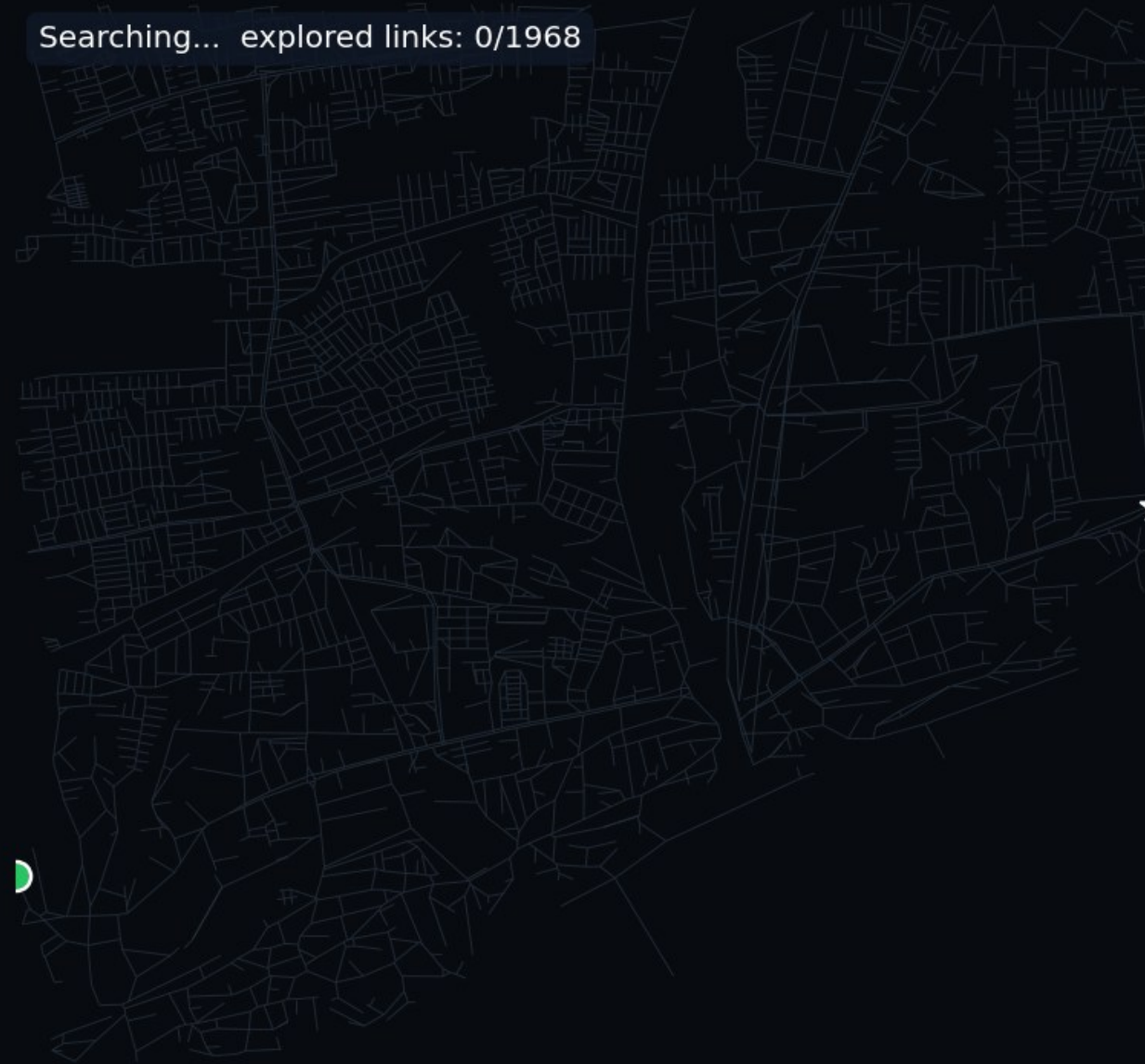
Winning Path Cost Calculation			
Step	From → To	Distance (km)	Cumulative Cost (g(n))
1	Arad → Sibiu	140	140
2	Sibiu → Rimnicu Vilcea	80	220
3	Rimnicu Vilcea → Pitesti	97	317
4	Pitesti → Bucharest	101	418
Total Cost (g) = 418 km			

☆ A* uses both the cost so far (g) and the estimated cost to goal (h) to guide the search and find the optimal (lowest cost) path.

— Chosen Path — Other Roads - - - Straight-line (Heuristic) (h(n)) ○ Start ○ Goal

ASTAR SEARCH ON REAL STREETS

Searching... explored links: 0/1968



Why A* Works So Well

Complete

Always finds a solution if one exists

Optimal

Finds the best possible solution

Efficient

Usually much faster than blind search

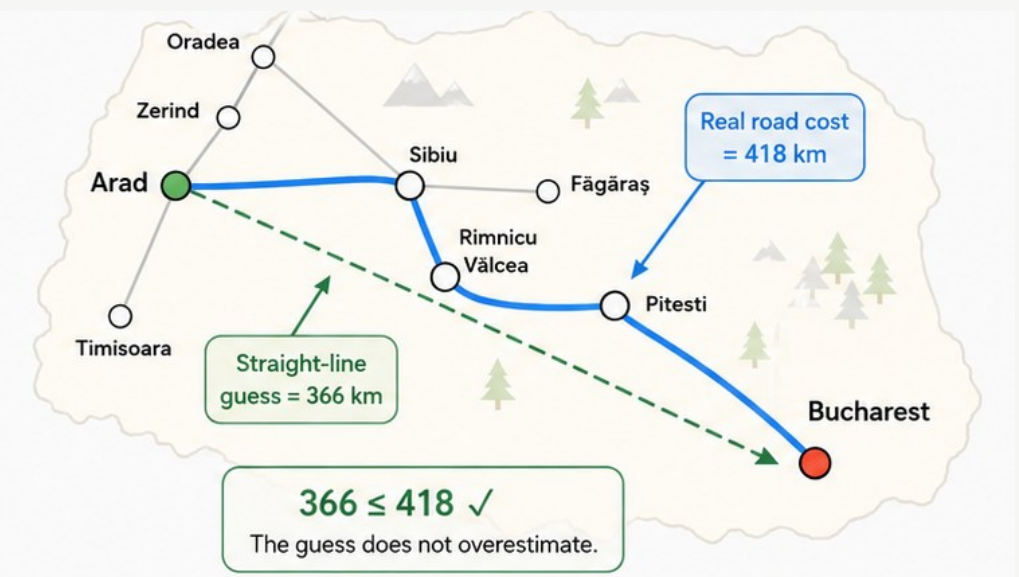
Memory-hungry

Can need a lot of memory

A "fair" heuristic

A heuristic must never overestimate the real cost — it can only guess low or exactly right.

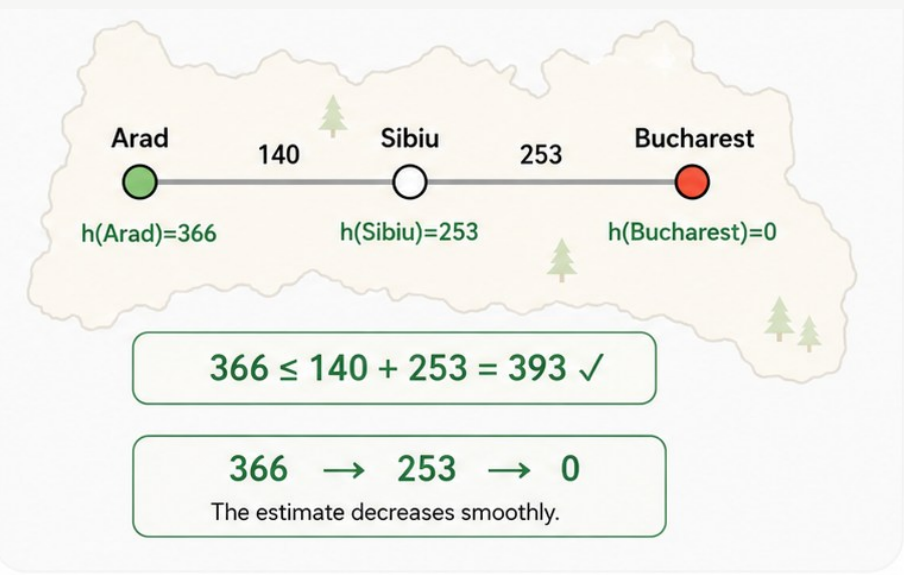
Straight-line Arad→Bucharest: 366 km. Real road distance: 418 km. Since $366 \leq 418$, it's a fair guess.



A "steady" heuristic

The guess can't jump around unfairly between neighboring cities — it has to change smoothly.

$h(\text{Arad})=366$. Cost to Sibiu=140, $h(\text{Sibiu})=253$. Check: $366 \leq 140+253=393$. ✓

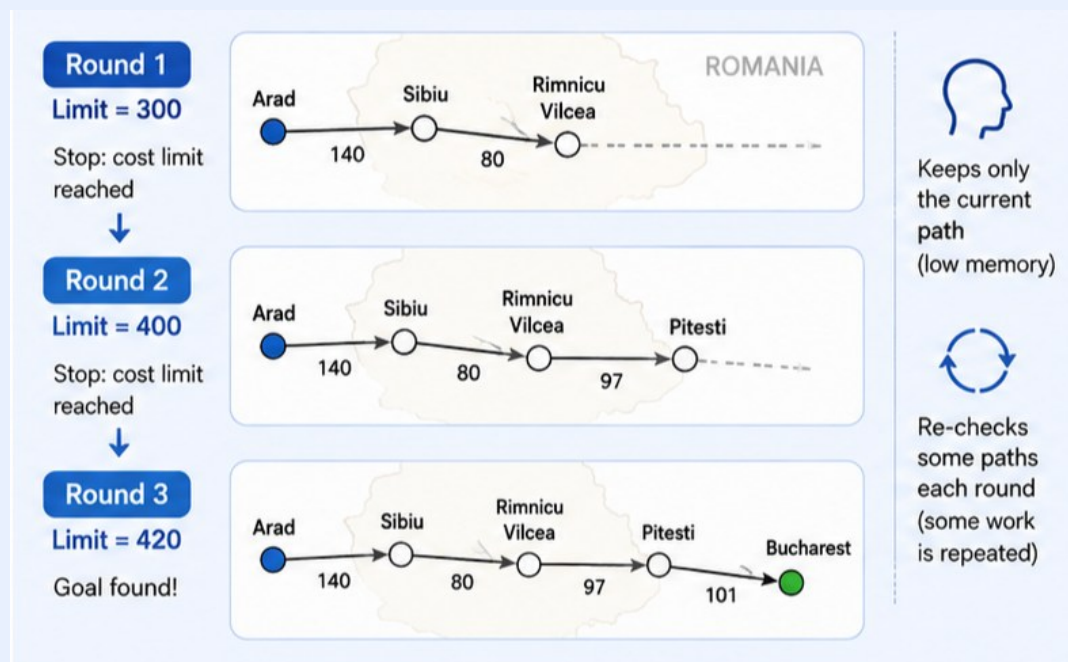


Two Handy Variations

IDA* — Iterative Deepening A*

Problem: regular A* remembers every path it has looked at, which can use a lot of memory.

Fix: search depth-first, but raise the allowed total cost a little each round until the goal turns up.

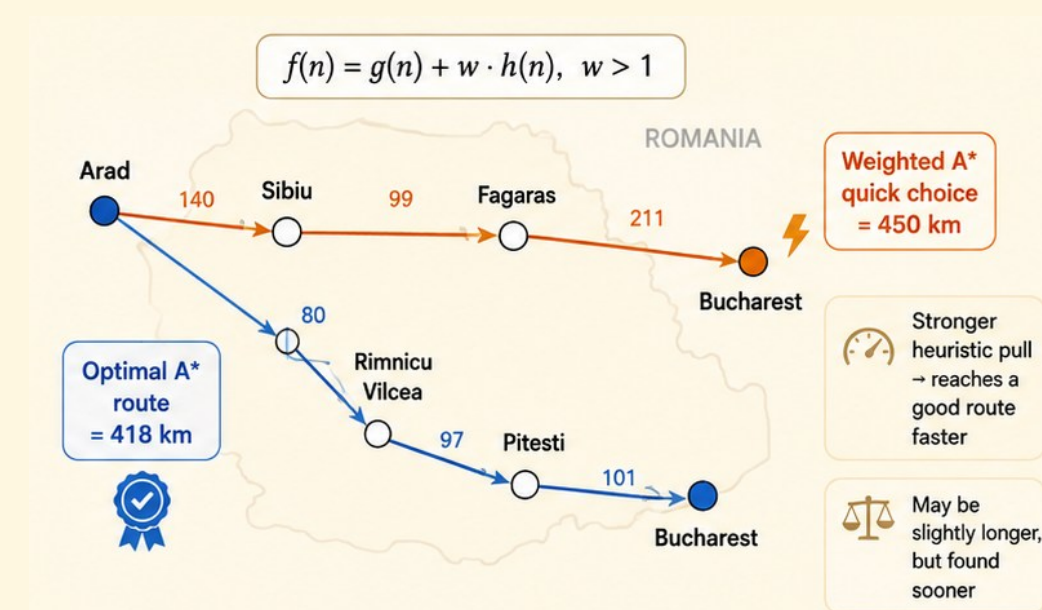


Uses far less memory, at the cost of re-checking some paths.

Weighted A*

Idea: multiply the guess $h(n)$ by a number greater than 1, so the search leans harder on its hunches.

Trade-off: faster, but the answer might not be perfectly the best anymore — just close to it.

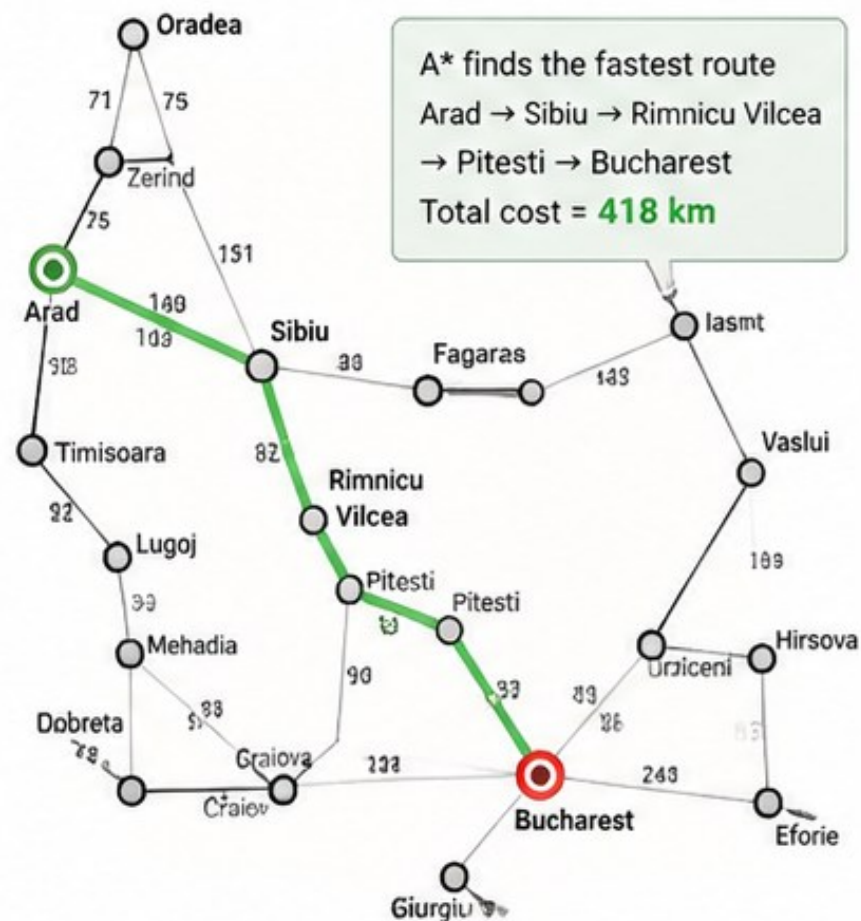


Many map apps use this for quick "good enough" routes when traffic keeps changing.

A* Is Already Around You

Map apps

Finding the fastest route
and avoiding traffic



Video games

Characters walking
around obstacles



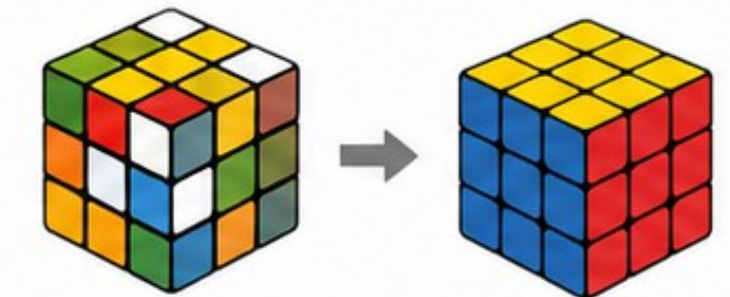
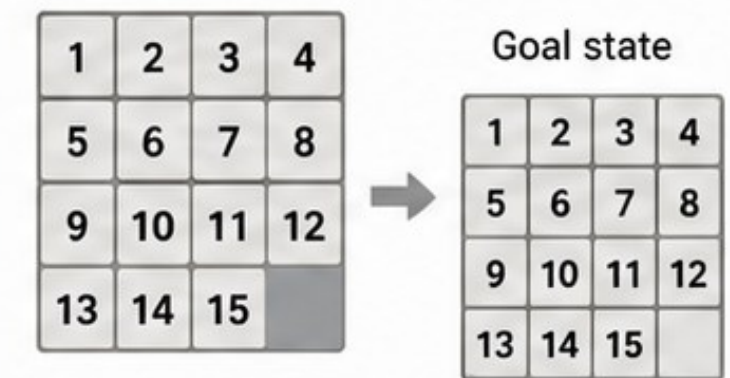
Robots

Warehouse robots and
self-driving cars



Puzzles

Solving a 15-puzzle or a
Rubik's Cube optimally



A* searches the shortest sequence
of moves to reach the solution.



NASA's Mars rovers use A*-style search to plan safe paths around rocks and obstacles on the Martian surface.